



1. SEMESTER-PROJEKT TANK!

PROJEKT GRUPPE SW4

NAVN

Frederik Okslund
Oskar Jentzsch Seeberg
Morten Rytter Rasmussen
Mikkel Jungclaus Therkildsen
Andreas Østrem Pihl
Viktor Kalhøj Sønderup
Tim Kanne Petersen

STUDIE NR.



Aarhus Universitet
Vejleder: [Redacted]
Dato: 20/06/2025

Indhold

Rapportskrivning.....	4
1. Problemformulering	5
2. Kravspecifikation	9
2.1. Indledning	9
2.2. Aktør-kontekst diagram	9
2.3. Funktionelle krav	10
Use case 1: ”Kør banen”	10
Use-case 2: ”Afspil lyd”	10
Use-case 3: ”Styr forlys”	11
Use case 4: ”Styr baglys”	11
Bilens Tema	11
2.4. Ikke-funktionelle krav	11
2.4.1. Generelle krav	11
2.4.2. Bilens forlys	12
2.4.3. Bilens bag- og bremselys	12
3. Systemarkitektur hardware	13
BDD og IBD	13
Signalbeskrivelse	15
4. Systemarkitektur software	16
Modulbeskrivelse	17
5. Hardwaredesign	19
Spændingsregulator	20
Motorstyring	22
Lys	24
Lyd	28
Driving Control	29
6. Softwaredesign	30
Motor	30
Lys	32
Lyd	33
Driving Control	35
7. Hardwareimplementering og modultest	37
Spændingsregulator	37
Motorstyring	38
Lys	39
Lyd	40

8. Softwareimplementering og modultest	43
Motor.....	43
Lys	45
Lyd.....	47
Driving Control	49
9. Integrationstest.....	52
10. Accepttestspecifikation	53
11. Konklusion.....	59
11.1. Konklusion for hardware delen.....	59
11.2. Konklusion for software delen.....	59
11.3. Konklusion for projektet.....	59
12. Individuelle konklusioner.....	60
13. Bilagsoversigt.....	63

Rapportskrivning

Denne tabel viser ansvarsfordelingen. Bemærk at ansvarlig(e) står for alt i processen fra HW/SW-Design til HW/SW-implementering og modultest.

- ☒ Ansvarlig(e)
☐ Fælles

Område	Fælles	FO	OJS	MRR	MJT	AØP	VKS	TKP
Forside	☐							
1. Problemformulering	☐							
2. Kravspecifikation	☐							
3. Systemarkitektur HW	☐							
4. Systemarkitektur SW	☐							
- Design af bil				☒				
5. Hardwaredesign								
6. Softwaredesign								
7. HW-implementering og modultest								
8. SW-implementering og modultest								
- Spændingsregulator			☒			☒		
- Motor			☒			☒		
- Lys		☒		☒				☒
- Lyd					☒		☒	
- Driving Control			☒					
9. Integrationstest	☐							
10. Accepttestspecifikation	☐							
11. Konklusion på projektet	☐							
12. Individuelle konklusioner	☐							
13. Bilagsoversigt	☐							

1. Problemformulering

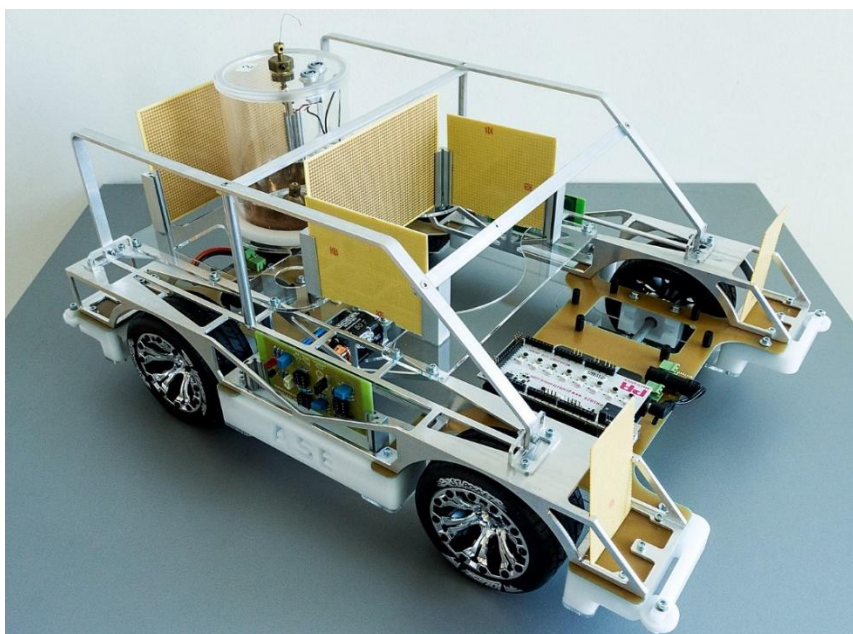
Det tværfaglige projekt PRJ1, der gennemføres på 1. semester for E- og SW-studierne, drejer sig om udvikling af hardware og software til styring af en elektrisk bil.

Som en indledning til projektet gennemføres 4 øvelser i E-LAB. Disse kan betragtes som en forundersøgelse eller analyse af de teknikker, der tænkes anvendt under projektet.

Projektet skal udarbejdes efter retningslinjerne i dokumentet "Vejledning til gennemførelse af projekt 1".

Under projektets forskellige faser gives der enkelte forelæsninger, der uddyber indholdet af disse faser.

Et bilkarrosseri med påmonteret motor bliver udleveret til hver enkelt projektgruppe. Se figur 1.



Figur 1. Karrosseri med printplader, pendul og Arduino Mega2560

Den færdige bils formål er, at den skal kunne gennemføre en konkurrencebane efter regler, der er nærmere beskrevet i bilag 1, "Konkurrencen".

Bilen skal udstyres med:

- Mikrocontroller.
- Elektronik til regulering af motor.
- Sensorer til detektering af refleksbrikker.
- For- og baglygter.
- Elektronik til styring af for- og baglygter.
- Elektronik og højttaler til afspilning af lyd.

Nogle af de nødvendige hardwareenheder vil helt eller delvis blive udviklet i andre kurser på 1. semester.

Arbejdet med disse opfattes derfor ikke som en del af semesterprojektet.

Det drejer sig om:

- Print med kredsløb til detektering af refleksbrikker.
Dette print skal forbindes til bilens mikrocontroller.
Hvis der foretages modifikationer af printet for at tilpasse det projektet, skal disse modifikationer beskrives i projektdokumentationen.
- ”Fejltæller”: Print med kredsløb til tælling og visning af pulser fra et pendul.
Pendulet anvendes kun som et påmonteret måleudstyr under konkurrencen, og den er ikke en del af selve projektet.

Bilen skal forsynes med fremadrettet kørellys (hvidt) og baglys (rødt), samt bremselys (kraftigt rødt).

Se figur 2 og figur 3.

Bremselys og baglys udsendes fra samme kilde og skal kunne reguleres i to forskellige styrker.

Hvert enkelt for-, bag- eller bremselys kan realiseres som et antal lysdioder, der opbygges som et LED-sæt.

Hvert sæt kan opbygges som en kombination af serie- og/eller parallel-forbundne lysdioder.

I projektet anvendes microcontrolleren Mega2560 på et ”Arduino Mega2560” kit.

Denne anvendes til:

- Styring af bilens fart og retning.
- Styring af for- og baglygternes lysstyrke.
- Detektering af banens refleksbrikker.
- Afspilning af lyde og/eller melodier.

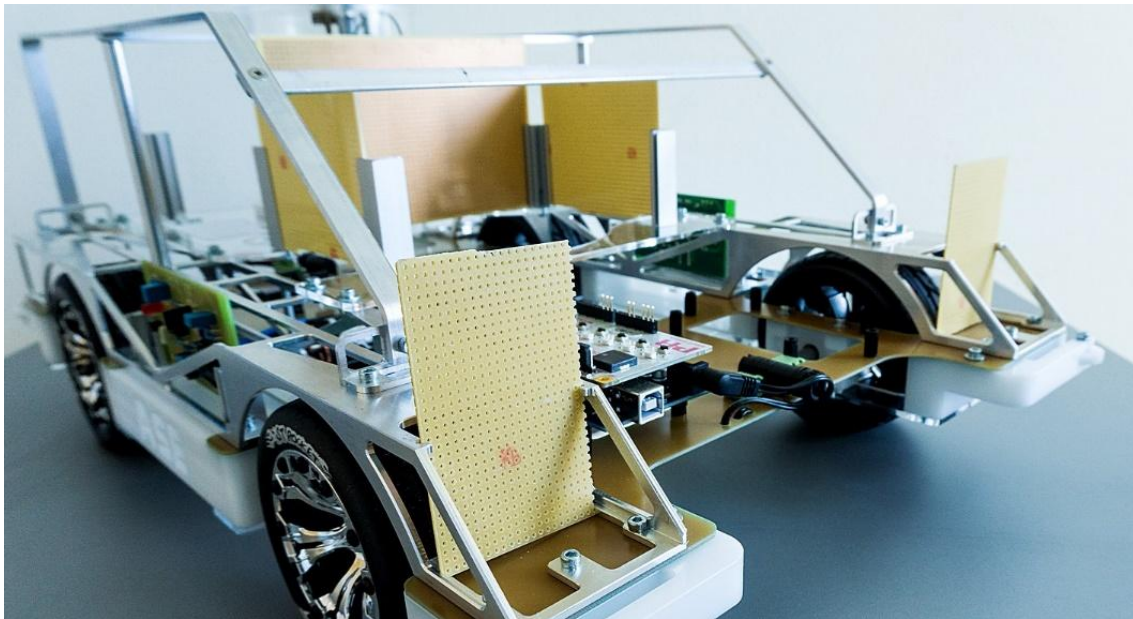
Bilen skal under konkurrencen køre på en bane med to baner, hvor banerne vil sikre, at bilen bliver på banen. Banerne er udstyret med 7 par refleksbrikker, der kan detekteres af optiske sensorer placeret på bilen.

Ved start skal bilen afspille en specifik startlyd eller startmelodi, og efter passage af mållinjen afspilles en specifik slutlyd eller slutmelodi. Ved passage af hver enkelt af refleksbrikkerne skal bilen afgive en specifik ”refleksbriklyd”.

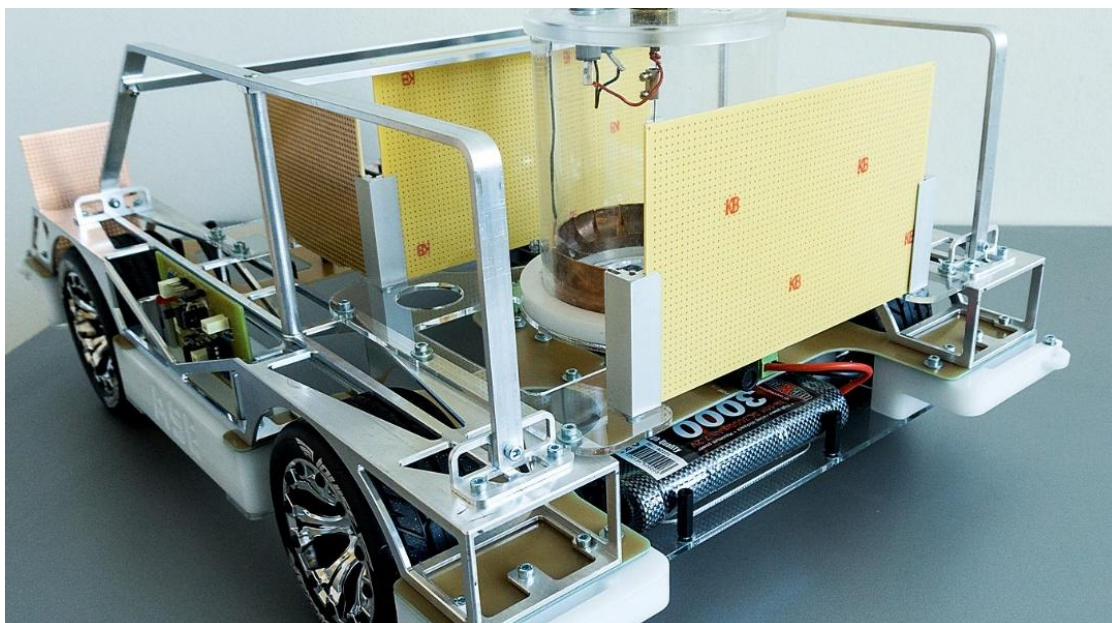
På figur 4 er vist placering af et af printene til detektering af refleksbrikker (der skal være et på hver side).

Figur 5 viser en mulig placering af print med anden elektronik, som skal konstrueres for at kunne styre bilen.

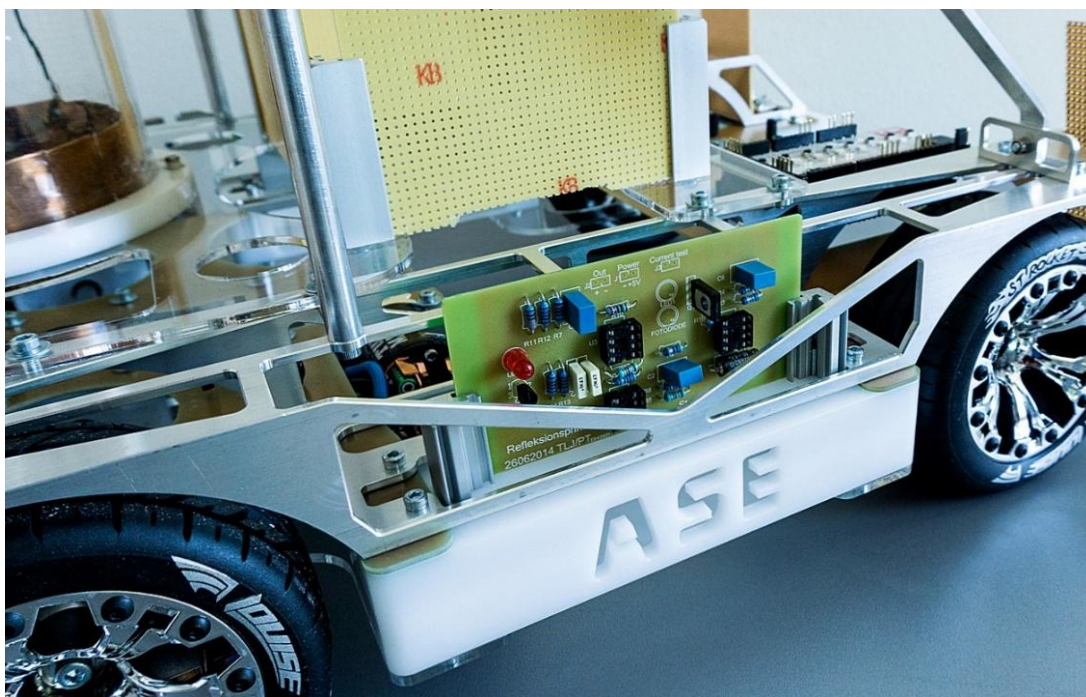
De præcise fysiske mål for bilkarrosseriet kan ses i bilag 1.



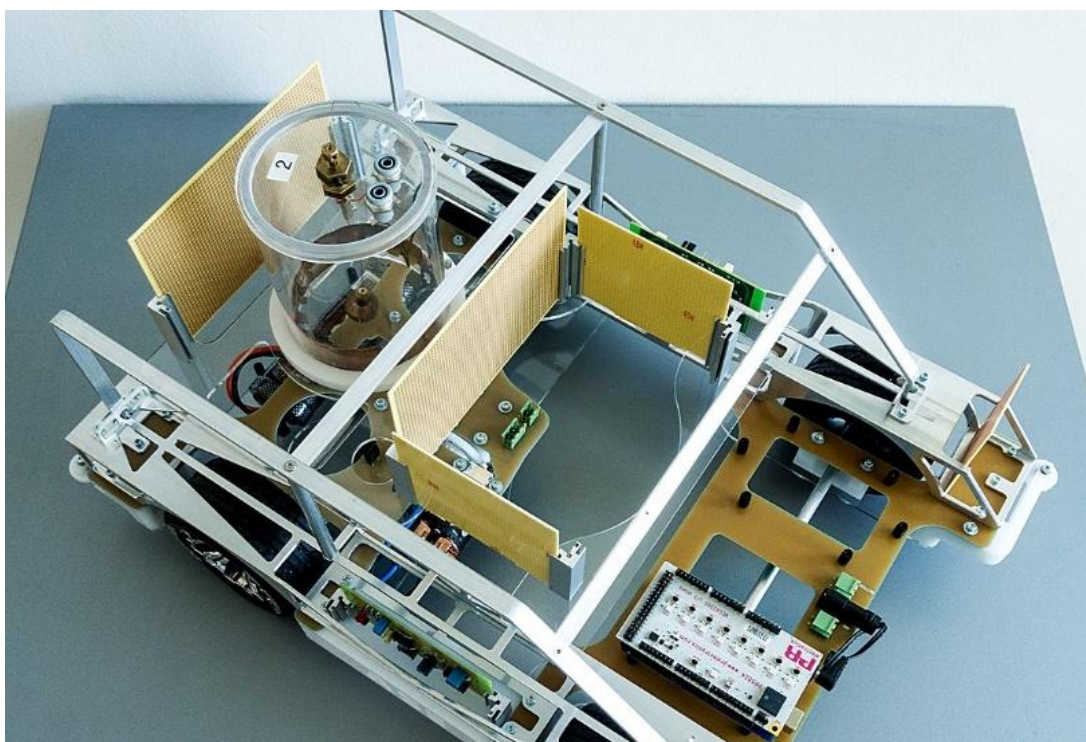
Figur 2. Placering af to print til forlys, højre og venstre side



Figur 3. Placering af print til bag- og bremselys



Figur 4. Placering af print til detektering af refleksbrikker



Figur 5. Print med øvrig elektronik placeret midt i bilen

2. Kravspecifikation

Udarbejdet af hele gruppen

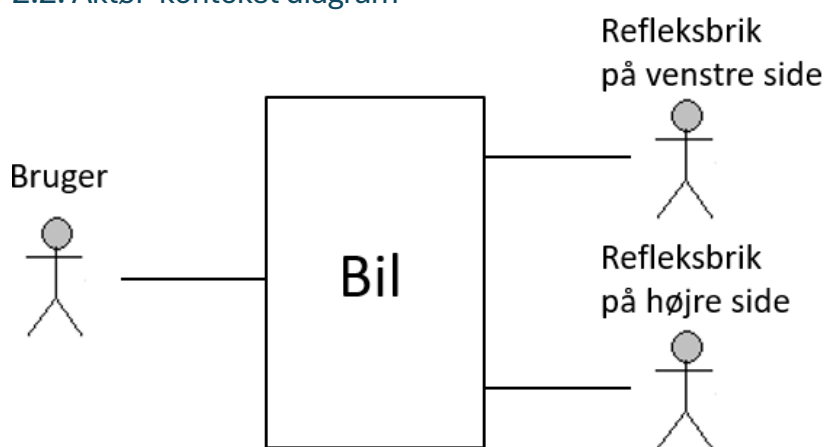
2.1. Indledning

Denne kravspecifikation beskriver de forskellige krav til det færdige projekt. Den forklarer de præcise krav, som forventes ved aflevering, samt indeholder den flere testbare krav.

Kravspecifikationen er dog modificerbar, hvorfor der kan forekomme ændringer. Der er sporbarhed i dette dokument – både forlæns og baglæns – hvorfor det altid vil være muligt at opspore, hvor kravet kommer fra, samt vil det være muligt at referere til et krav.

Dette dokument indeholder to overpunkter, "Funktionelle" og "Ikke-funktionelle" krav. De funktionelle krav beskriver de specifikke funktionaliteter projektet skal have, hvorimod de ikke-funktionelle krav beskriver de generelle tværgående krav.

2.2. Aktør-kontekst diagram



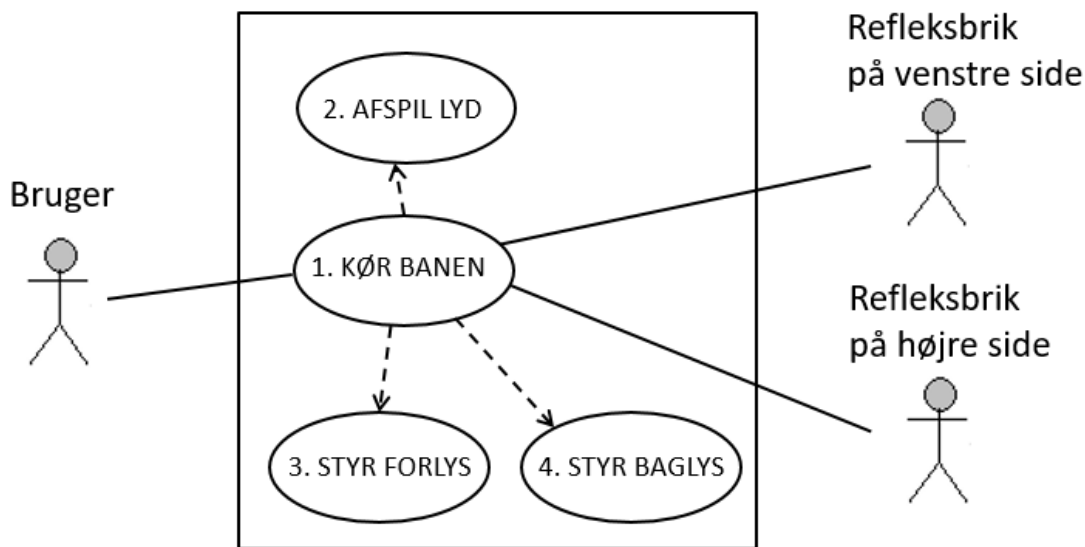
Figur 6. Aktør-kontekst diagram

Det system, der skal udvikles, er den elektriske bil. Figur 6 illustrerer systemet "Bil" samt de aktører, der interagerer med den.

Den primære aktør, "Bruger", kan starte bilens gennemkørsel af banen.

Derudover fungerer "Refleksbrik på venstre side" som en sekundær aktør, da bilen registrerer den, hver gang den passerer. Det samme gælder for "Refleksbrik på højre side", som bilen ligeledes detekterer under kørsel.

2.3. Funktionelle krav



Figur 7. Use case diagram

Figur 7 viser de "Use Cases", som bilen skal kunne udføre, samt de aktører, der er involveret.

Use case 1: "Kør banen"

Mål

Denne use case beskriver, hvordan banen skal gennemkøres af denne bil.

Initieres af: Bruger.

Normalt scenarie

1. Brugeren tænder for bilen og placerer den, så den er klar til at køre forlæns ind på banen.
2. Brugeren trykkes på start, så bilen kører forlæns ind på banen ved at passere startlinjen.
3. Når bilen har passeret refleksbrik nummer 6, bringes bilen til standsning, inden refleksbrik nummer 7 nås.
4. Bilen bakker, indtil refleksbrik nummer 5 er passeret, og bringes til standsning, inden refleksbrik nummer 4 nås.
5. Bilen kører forlæns, indtil refleksbrik nummer 7 nås.
6. Bilens bringes til standsning helt inden for mål området, der er defineret som området mellem refleksbrik 7 og en meter efter refleksbrik 7

Generelt gælder også:

1. Lyd afspilles som beskrevet i use-case 2 "Afspil lyd".
2. Forlyset styres som beskrevet i use-case 3 "Styr forlys".
3. Baglyset styres som beskrevet i use-case 4 "Styr baglys".

Use-case 2: "Afspil lyd"

Mål

Denne use-case beskriver styring af en i systemet indbygget højttaler.

Initieres efter: Use-case 1, og sammen med use-case 3 og 4.

Normalt scenarie

1. Når bilen tændes for at køre ind på banen, afspilles en specifik "startlyd" eller "startmelodi", som er prædefineret. (Se punkt 2.4.4.1 i "Ikke-funktionelle krav").

2. Hver gang en refleksbrik passeres, afspilles en specifik ”refleksbriklyd”, som er prædefineret. (Se punkt 2.4.4.2 i ”Ikke-funktionelle krav”).
3. Når refleksbrik nummer 7 passeres, afspilles en specifik ”slutlyd” eller ”slutmelodi”, som er prædefineret. (Se punkt 2.4.4.3 i ”Ikke-funktionelle krav”).

Use-case 3: ”Styr forlys”

Mål

Denne use case beskriver styringen af bilens indbyggede forlys. Forlyset kan styres til at være slukket eller tændt.

Initieres efter: Use case 1, og sammen med use case 2 og 4.

Normalt scenarie

1. Forlyset skal være tændt, når bilens motor påtrykkes en spænding, for at bringe den til at køre. Både forlæns og baglæns
2. I alle andre tilfælde, skal forlyset være slukket.

Use case 4: ”Styr baglys”

Mål

Denne use case beskriver styringen af bilens indbyggede baglys. Baglyset kan være slukket, lyse med mellemstyrke (”almindeligt baglys”) eller lyse med kraftig styrke (”bremselys”).

Initieres af: Use case 1, og sammen med use case 2 og 3, dog afhængig af situationen.

Normalt scenarie

1. Baglyset skal lyse med mellemstyrke (”almindeligt baglys”), når bilens motor påtrykkes en spænding, for at bringe den til at køre. Er bilen ikke påtrykket en spænding, skal baglyset være slukket.
2. Baglyset skal lyse med kraftig styrke (”bremselys”), mens spændingen til bilens motor mindskes.
3. Baglyset skal herefter lyse kraftigt i 0,5 sekunder +/- 0,1 sekunder.

Bilens Tema

1. Bilen kunne ligne en Panzer VIII Maus Tank
2. Bilen kunne have en krigs-inspireret sang som startlyd
3. Bilen kunne have et kanonskud som reflekslyd
4. Bilen kunne have en sejrsmelodi som slutlyd

2.4. Ikke-funktionelle krav

2.4.1. Generelle krav

1. Bilen skal på en enkelt opladning af dennes batterier kunne gennemføre mindst 5 gennemkørsler af banen.
2. På bilens højre og venstre side skal placeres detektorer, der kan registrere en R80 refleksbrik i afstanden 2 cm til 25 cm. Se figur 4 og figur 6.
3. Bilen monteret med al udstyr må maksimalt veje 5,2 kg.
4. Bilens maksimale højde skal være 41 cm.
5. Bilens maksimale bredde skal være 38 cm.

2.4.2. Bilens forlyst

1. Implementeres med 2 hvide LED-sæt, der monteres med et sæt i henholdsvis højre og venstre side. Se figur 2.
2. Når forlyset er tændt, skal det kunne ses på en afstand på minimum to meter
3. Når forlyset er tændt, skal middelstrømmen gennem hvert LED-sæt være 50 mA +/- 5 mA.

2.4.3. Bilens bag- og bremselyst

1. Implementeres med 2 røde LED-sæt, der monteres med et sæt i henholdsvis højre og venstre. Se figur 3.
2. Når bag- og bremselyset er tændt, skal det kunne ses på en afstand på minimum to meter.
3. Ved ”bremselyst” skal middelstrømmen gennem hvert LED-sæt være 50 mA +/- 5 mA.
4. Ved ”almindeligt baglyst” skal middelstrømmen gennem hvert LED-sæt være 10 mA +/- 2 mA.

2.4.4. Bilens lyd

1. Startlyden kan spille kontinuerligt i 2 sekunder (eller indtil melodien er slut)
2. Startlyden burde være hørbar fra 5 meters afstand og må ikke afbrydes af andre lyde.
3. Afspilning af reflekslyd burde kunne påbegyndes indenfor 100 ms af registrering af refleksbrik
4. Reflekslydens niveau kan måles til 55 dB på 5 meters afstand.
5. Slutlydens niveau kan måles til 60 dB på 5 meters afstand.
6. Slutlyden burde ikke afbrydes (eksempelvis af refleksbrik-lyden)

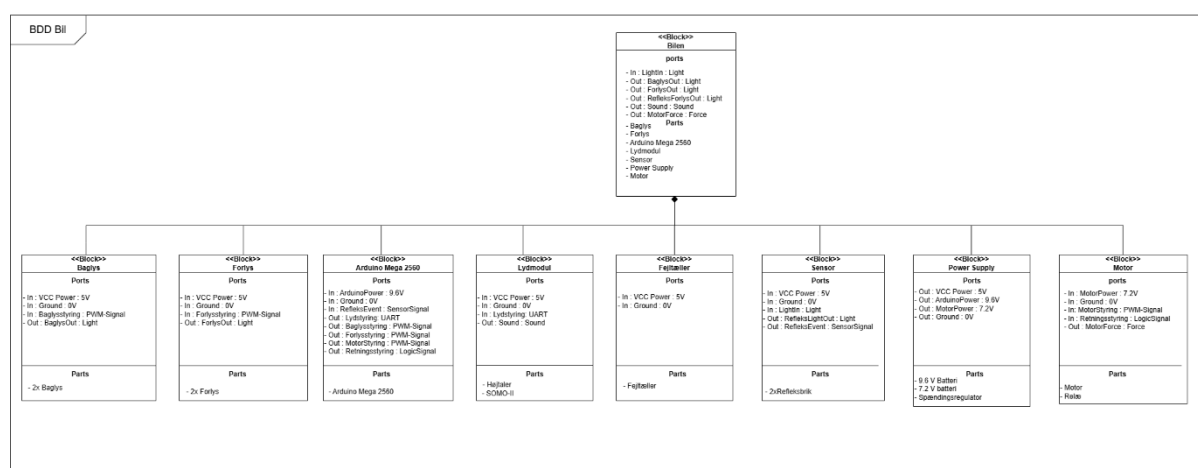
3. Systemarkitektur hardware

Udarbejdet af hele gruppen

BDD og IBD

Hardwarearkitekturen har til formål at beskrive hvordan bilens forskellige hardwaredele interagerer med hinanden. Hardwarearkitekturen er vigtigt, da den er med til at overskueliggøre en ellers stor opgave ved at bryde den ned i mindre dele. Det giver et overblik, som er nødvendigt inden det egentlige arbejde kan påbegyndes. For at visualisere hardwarearkitekturen, er der blevet udarbejdet et BDD (Block definition diagram) og et IBD (Internal Block Diagram).

BDD bruges til at definere systemet på et højt niveau og vise relationerne mellem funktionsblokkene og deres forbindelser. Det viser også hvilke fysiske komponenter, der skal samarbejde, og fungerer som systemets overordnede struktur, se Figur 8.



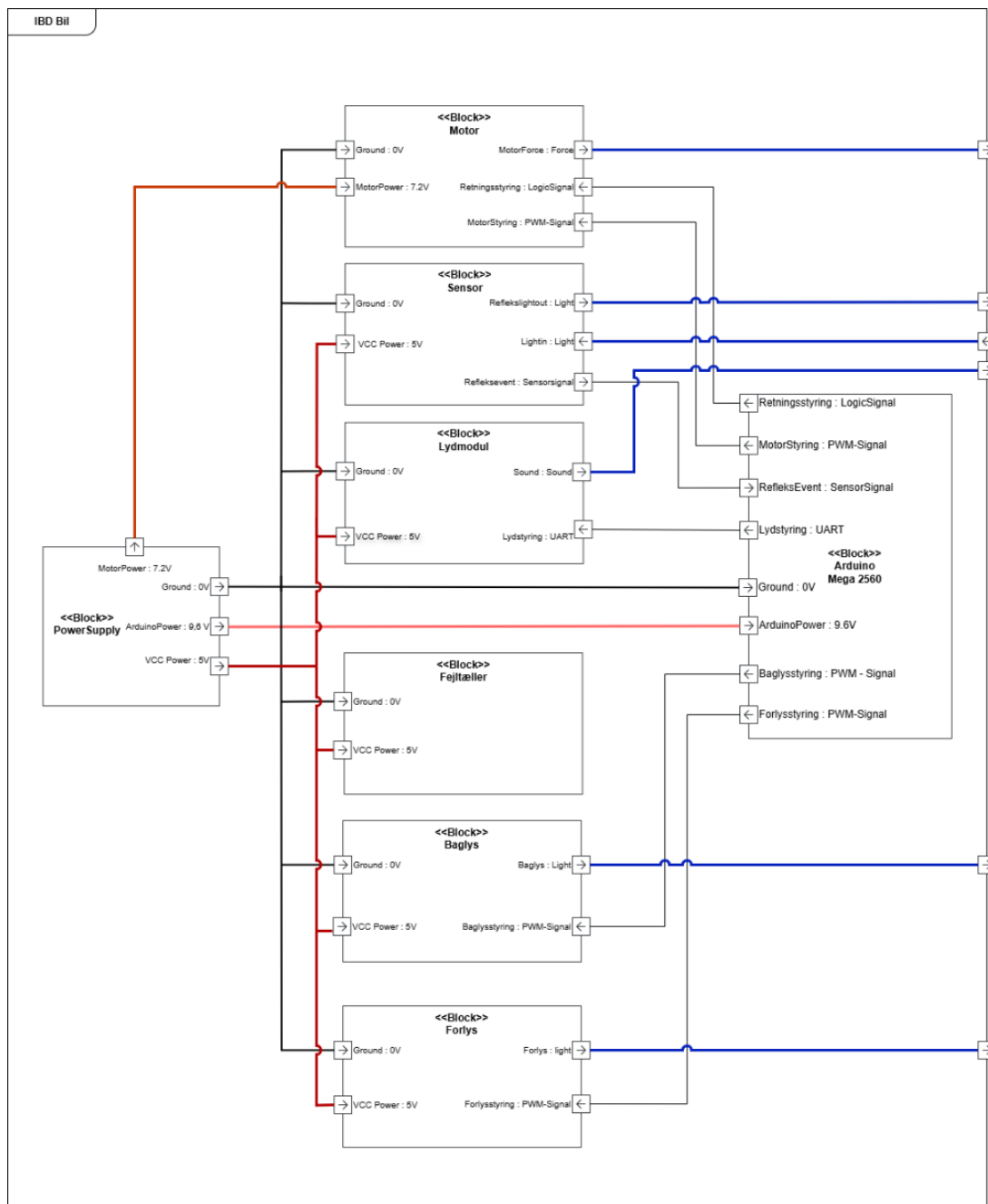
Figur 8. Block Definition Diagram over bilens hardware

Da det ikke er muligt udelukkende at beskrive blokkene igennem BDD-diagrammet laves der en blokbeskrivelse, som er vist nedenfor, se Tabel 1

Blok	Funktionalitet
Baglys	Styrer baglyset
Forly	Styrer forlyset
Arduino Mega 2560	Processerer input fra Sensor
Lydmodul	Styrer lyden
Fejltæller	Tæller fejl ved for hurtig acceleration
Sensor	Detekterer lys, som reflekteres fra en refleksbrik
Power Supply	Forsyner Sensor og Arduino Mega2560 med spænding
Motor	Får bilen til at køre

Tabel 1 - Beskrivelse af blokkene

IBD viser hvordan hver enkelt blok internt hænger sammen, og viser de fysiske forbindelser mellem blokkene. Derudover viser diagrammet retningen på signalerne, deres navne samt deres type og størrelse, se Figur 9.



Figur 9. Internal Block Diagram over bilens hardware

Signalbeskrivelse

Da det ikke er muligt fuldstændigt at beskrive signalerne igennem et BDD og et IBD beskrives først blokkene i større detalje og dernæst signalerne, deres navne, type, og blokkene i større detalje i tabellen nedenfor, se Tabel 2. Derudover er der tilknyttet en kommentar.

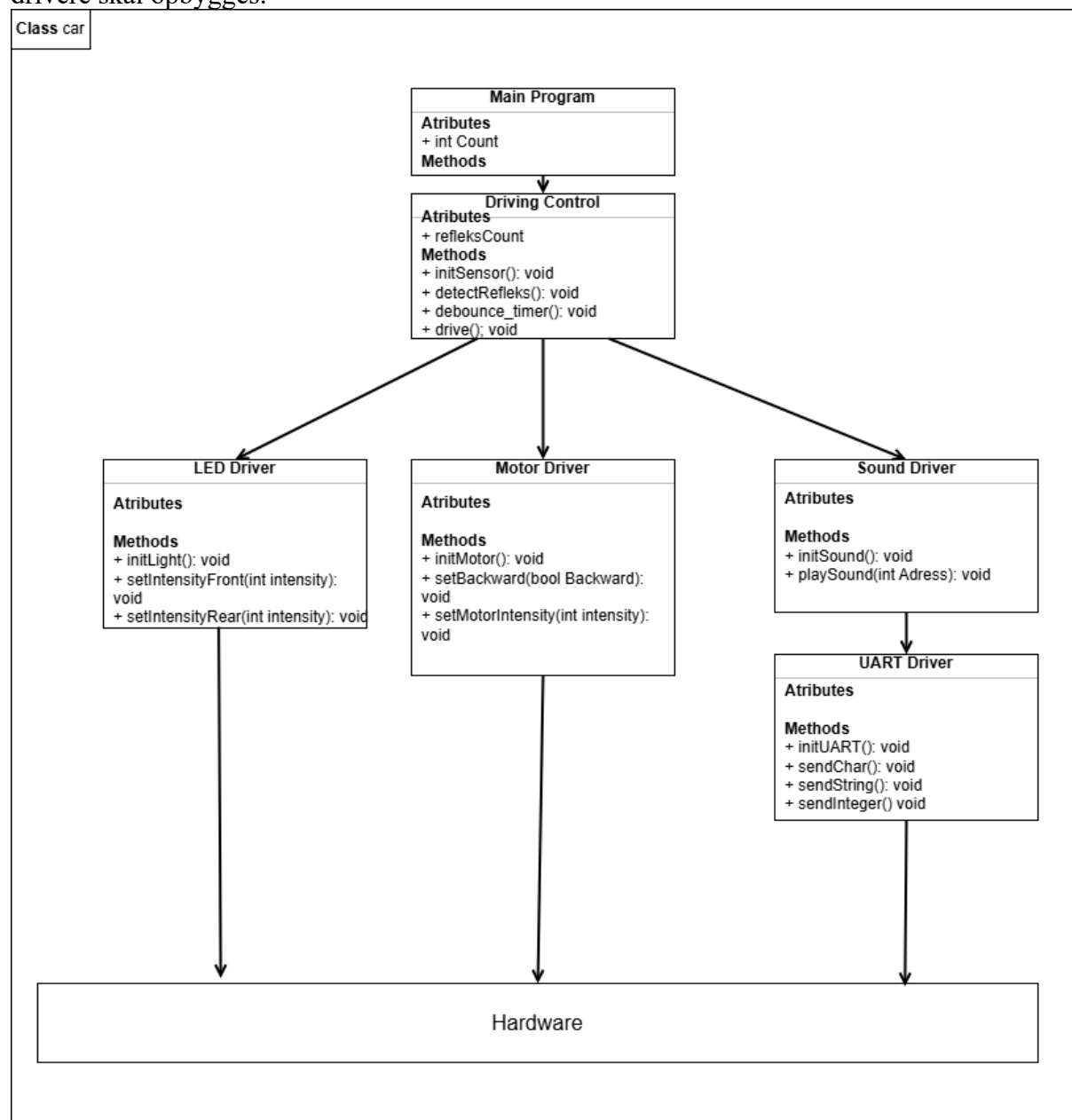
Signalnavn	Funktion	Område	Port 1 (source)	Port 2 (destination)	Kommentar
ground	Reference til analog spænding	0V	Power supply, ground	Arduino Mega2560	Stel
Arduino power	Forsyningsspænding	7V-12V	Power supply	Arduino Mega2560	
VCC power	Forsyningsspænding	5V	Power supply	Motor, Sensor, Lydmodul, Fejltæller, Baglys, Forlys	
Motor-power	Forsyningsspænding	7.2V	Power supply	Motor	
MotorForce					
Retningsstyring	Accelerationens retning	Logic signal	Arduino Mega2560	Motor	
Motorstyring	Accelerationens størrelse	PWM signal	Arduino Mega2560	Motor	
Reflekslightout	Lys indikerer detektering af refleksbrik		Sensor		
LightIn	Lys reflekteret fra refleksbrik		Sensor		
RefleksEvent	Signal til Arduino ved refleks-detektering	0V-5V	Sensor	Arduino Mega2560	SensorSignal
Sound	Lyd		Lydmodul		
Lydstyring	Signalerer initiering af lydafspilning	UART	Arduino Mega2560	Lydmodul	
Baglys	Lys		Baglys		
Baglysstyring	Signal fra Arduino dikterer lys-intensitet	PWM signal	Arduino Mega2560	Baglys	
Forlys	Lys		Forlys		
Forlysstyring	Signal fra Arduino dikterer lys-intensitet	PWM signal	Arduino Mega2560	Forlys	
Fejltæller	Registrer fejl	0.5V	Ground, VCC power	Fejltæller	

Tabel 2 - Viser en udvidet beskrivelse af signalfunktionaliteten

4. Systemarkitektur software

Udarbejdet af alle i gruppen

Softwarearkitekturen har til formål at beskrive hvordan bilens forskellige softwaredele interagerer med hinanden. Det bruges som et blueprint til at opbygge softwaren, og forholder sig til kommunikationen mellem softwaren på et tidligt stadie, som gør designdelen lettere. Softwaren er blevet opdelt i ansvarsområder, der relaterer sig til en hardwareenhed, og kaldes derfor drivere. Den overskueliggør processen på samme måde som hardwarearkitekturen. Til at visualisere hvordan driverne kommunikerer og deres interne forhold, er der udarbejdet et UML-diagram, se Figur 10. UML-diagrammet hjælper med at gøre softwaren mere konkret, samtidig med at give et overblik over den ønskede funktionalitet og viser hvordan de enkelte drivere skal opbygges.



Figur 10. Unified Modeling Language diagram over bilens software

For at give et indblik ind i funktionaliteten bag hver enkelt funktion i hver enkelt modul, er der en kortfattet modulbeskrivelse nedenfor.

Modulbeskrivelse

Modulbeskrivelse af 'Driving Control'

Ansvar: Detektere reflekser, og styrer kørslen i overensstemmelse med antallet af reflekser.

Funktioner:

`void initSensor( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Initierer sensoren

`Void debounceTimer( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Aktiverer detectrefleks efter en given mængde tid

`Void detectRefleks( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Detektere refleksbrikker, hvorved count inkrementeres, og slukker sig selv

`Void drive( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Styrer intensiteten af lysene, intensiteten og retningen af motoren ved de forskellige refleksbrikker.

Modulbeskrivelse af 'LED Driver'

Ansvar: Styrer for og baglys, og deres intensitet.

Funktioner:

`void initLys( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Initierer LEDerne

`void setIntensityFront(int intensity)`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Styrer forlyset og dets intensitet

`void setIntensityRear(int intensity)`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Styrer baglyset og dets intensitet

,

Modulbeskrivelse af 'Sound Driver'

Ansvar: Styrer lydens volumen og hvad der afspilles.

Funktioner:

`void initSound( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Initierer lyden.

`void PlaySound(int sound)`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: styrer hvilken lyd der skal spilles.

Modulbeskrivelse af 'UART Driver'

Ansvar: At implementere en UART-driver, så man kan sende og modtage data via seriel kommunikation.

Funktioner:

`void InitUART( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Initierer UART-driveren.

`void SendChar( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Sender tegn via UART

`void SendString( )`

 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: Sender en streng via UART

`void SendInteger( )`

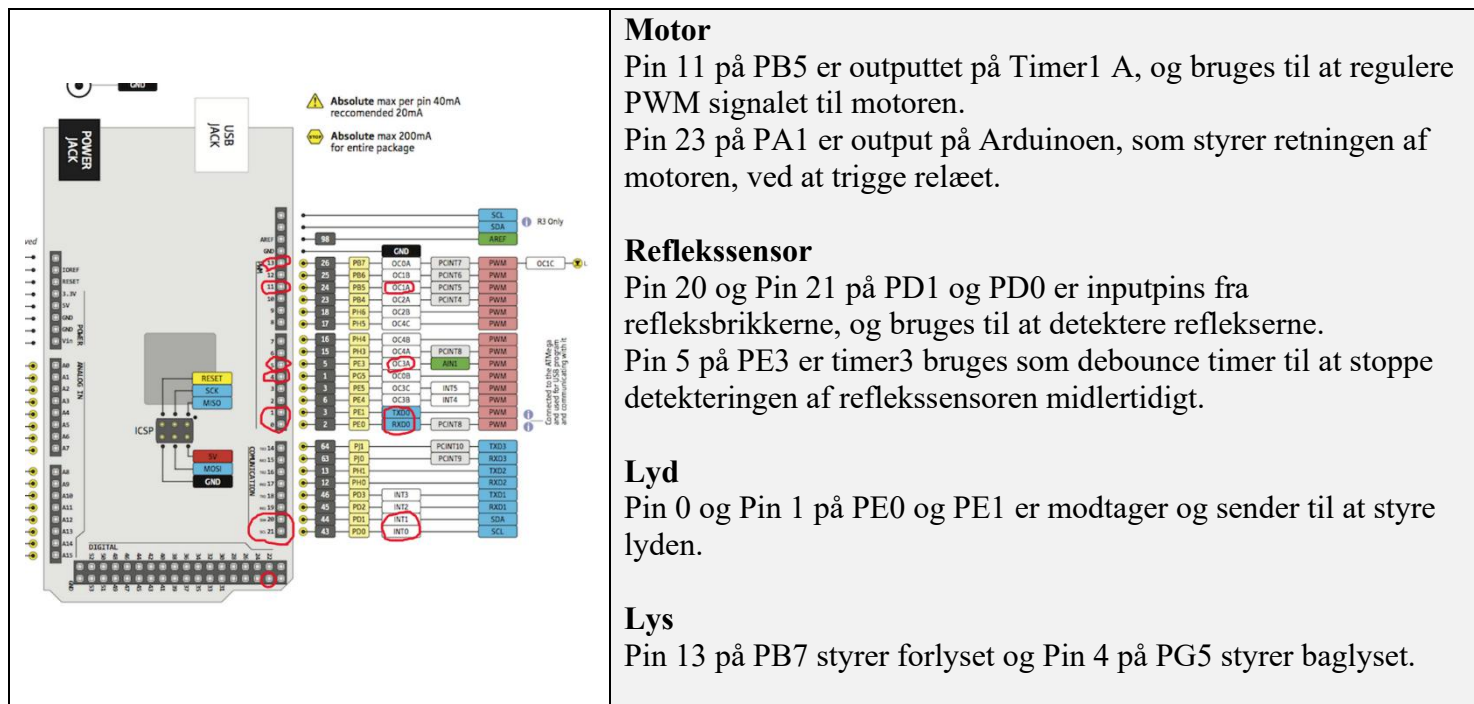
 Parametre: ingen

 Returværdi: ingen

 Beskrivelse: konverterer tal til en streng, og sender via UART.

5. Hardware design

Formålet med hardware design er at planlægge hardware delen af bilen, så det understøtter den funktionalitet, der er beskrevet i kravspecifikationen. Hver enkelt afsnit er udarbejdet med fokus på de valg der ligger bag hardware designet, og beskrevet så det er muligt at implementere. Nedenstående figur viser et overblik over forbindelserne til Arduinoen.



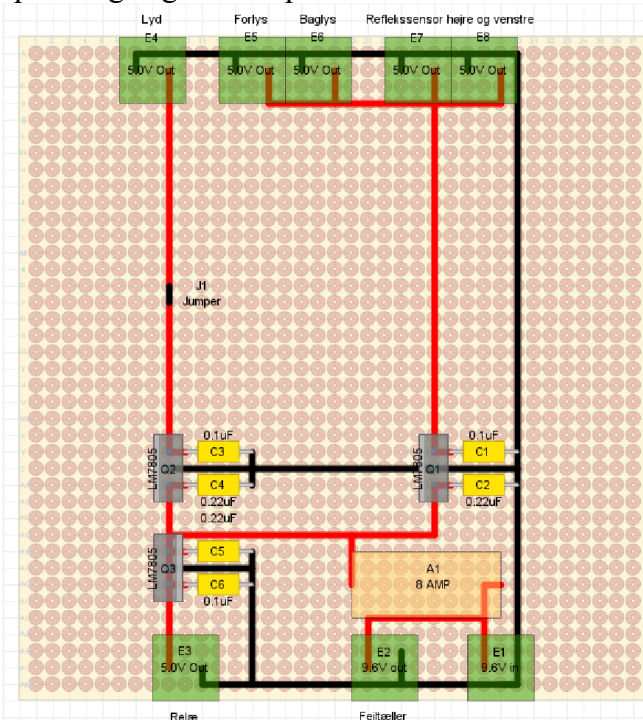
Figur 11. Overbliksbilleder over forbindelserne til Arduinoen

Udarbejdet af Andreas & Oskar

Mange af bilens enheder: fejltælleren, relæet, lyset og højtaleren skal bruge 5V, og uden en spændingsregulator vil disse enheder blive overbelastet af bilens batteri på 9,6V. For at undgå dette konstrueres et Veroboard med spændingsregulatorerne, af typen LM7805, der regulerer forsyningsspændingen ned til 5,0V. Ifølge databladet bruges spændingsregulatoren som vist på nedenstående figur:

The diagram shows a 5V regulator circuit. A 9.6V battery (Vbat1) is connected to the input of the LM7805CT regulator (U1). A 0.22µF capacitor (C1) is connected between the input and ground. The regulator's output is connected to a 0.1µF capacitor (C2) and ground. The output voltage is 5V. The current is labeled 1_AMP. The regulator is labeled U1 and LM7805CT. The output is labeled 5V.

Spændingsregulatoren sikrer en stabil spænding. Samtidig forsynes spændingsregulatoren af det store 9.6V batteri, da motorbatteriet under belastning kan medvirke et fald i outputspænding. Eftersom vi skal bruge 5,0V til flere af komponenterne opbygges 3 spændingsregulatorer på et Veroboard i overensstemmelse med nedenstående figur.



20

Som det også fremgår af Figur 13, er der terminaler til at trække strøm ud til diverse komponenter, Lyd, Forlys, Baglys, Reflekssensor højre og venstre, samt relæet til motoren. Styklisten tager udgangspunkt i Veroboard diagrammet.

Stykliste til spændingsregulatorerne

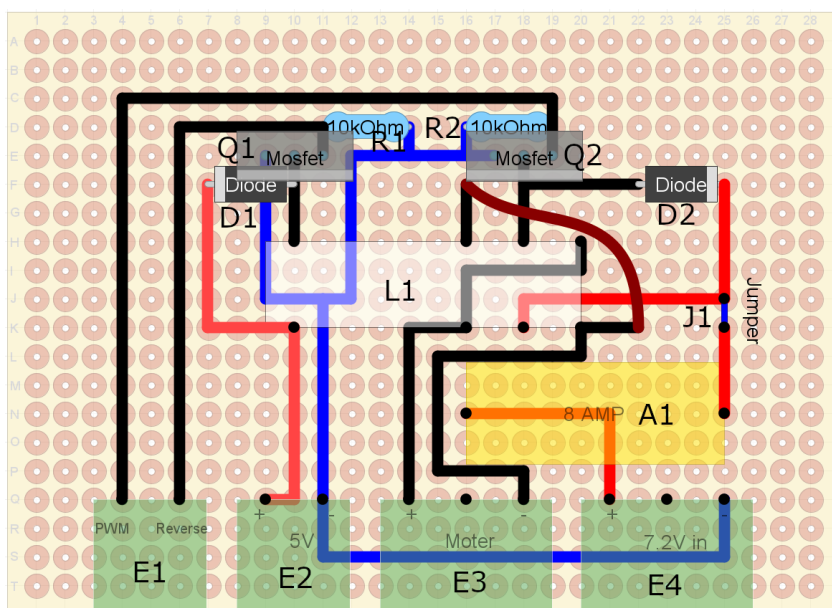
Antal	Symbol på diagram	Komponent	Type
8	E1, E2, E3, E4, E5, E6, E7, E8	Euroblok 2 stik	
1	A1	Sikring	8 AMP
1		Sikringsholder	
3	C1, C3, C6	Film kondensator	0,1 uF
3	C2, C4, C5	Film kondensator	0,22 uF
3	Q1, Q2, Q3	Spændingsregulator	LM7805
1	J1	Jumper	

Tabel 3. Stykliste til Veroboardet med spændingsregulatorerne

Afhængigt af dutycyclen på PWM-signalet løber der en strøm igennem motoren, som bliver tilført fra 7,2V batteriet. Et diagram, der viser dette er vist nedenfor på figur 14.



22



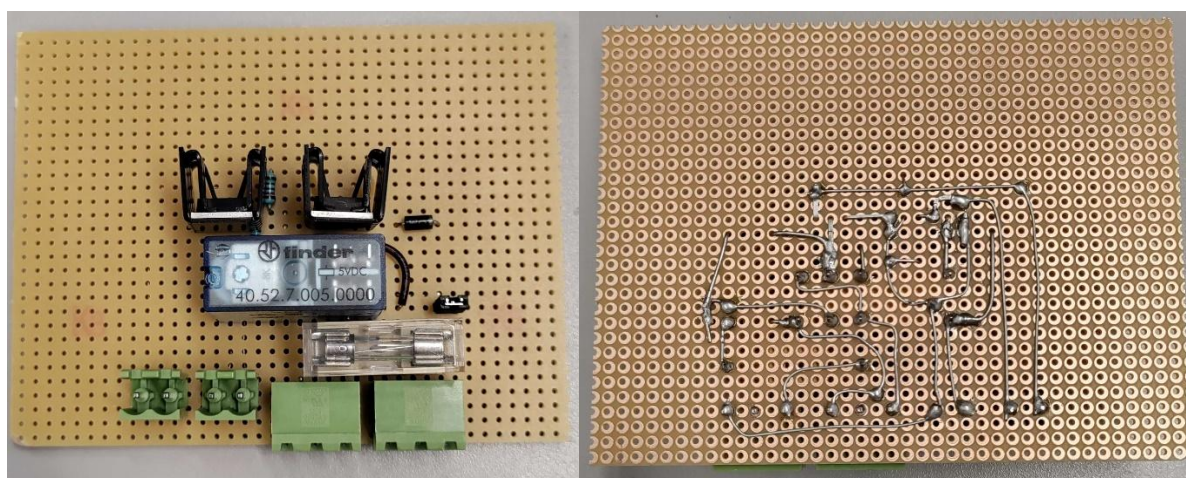
Figur 15. Veroboard diagram til motorstyring

Stykliste

Styklisten tager udgangspunkt i Veroboard-diagrammet til motorstyring på Figur 15.

Antal	Symbol på diagram	Komponent	Type
2	E1, E2	Euroblok 2 stik	
2	E3, E4	Euroblok 3 stik	
1	A1	Sikring	8 AMP
1		Sikringsholder	
2	R1, R2	Modstande	10 kΩ
1	J1	Jumper	
2	Q1, Q2	Mosfet Transistor	IRLZ44
2	D1, D2	Diode	1N4007
1	L1	Relæ	40.52.7.005.0000

Tabel 4. Stykliste til veroboardet med motorstyring



Figur 16. Forside og bagside af veroboardet med motorstyring

Formål

Formålet med lyset er at være tændt under kørslen, lysets intensitet styres af en Arduino ATmega2560. Forlyset skal lyse med en strøm på 50mA under hele kørslen og baglyset med en strøm på 10mA under frem ad kørsel og en strøm på 50mA under bremsning/ baglæns kørsel. Strøm og spænding leveres af batteriet på 9.6V som bliver reguleret af spændingsregulatoren til en spænding på 5V. Dette løber igennem en mosfet (IRLZ44N), som afhængigt af dutycyclen af PWM-signalet sender den korrekte strøm igennem LEDerne.

På bilen er monteret to separate lyssystemer – For- og baglys.
Disse styres ved en kontrolleret strøm på:

Forlys	50mA +/- 5mA
Baglys	10mA +/- 2mA
Bremselys	50mA +/- 5mA

For at styre strømmen der løber igennem hvert sæt anvendes en Arduino ATmega2560. Denne sender to separate PWM-signaler til hhv. forlys og baglys. Kredsløbene til både forlys og baglys er konstrueret ens; den eneste forskel er for-modstandene som anvendes til LEDerne idet spændingsfaldet over de røde LEDer er mindre end over de hvide LEDer. x

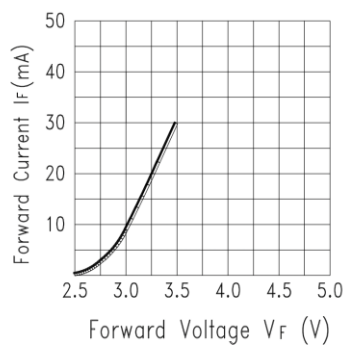
Forlys:

$$R_{Forlys} = \frac{VCC - V_{LED,hvid}}{25mA}$$
$$R_{Forlys} = \frac{5V - 3.375V}{25mA} = 65\Omega$$

I MultiSim-diagrammet navngiver vi disse modstande som $R_1 - R_4$, og vi anvender desuden nogle på 68Ω i praksis. Spændingsfaldet henover LEDen på $3.375V$ findes blot ved at kigge på databladet for den hvide LED (LTW-2S3D8). Spændingsfaldet hen over mosfet (IRLZ44N) er så småt at vi ignorerer det i udregningen.

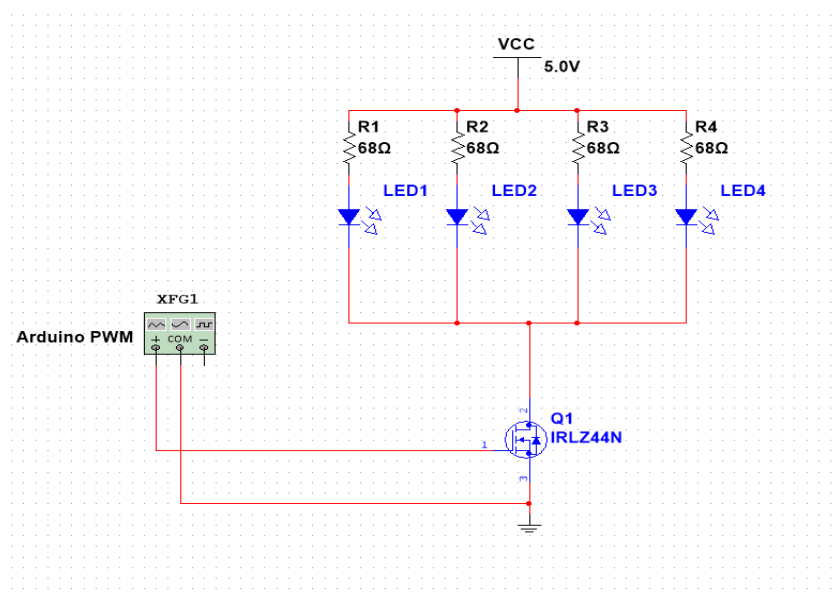
De $25mA$ er strømmen igennem hver af de to LEDer (forbundet i parallel) da vi som beskrevet i kravspecifikationen sender en strøm på $50mA$ igennem hvert sæt af lygter.

På vero-boardet er desuden monteret et par Harwin-pins med en jumper der tillader måling af strøm igennem én enkelt af LEDerne; dette er for at kunne bekræfte at kredsløbet opfylder vores kravspecifikation.

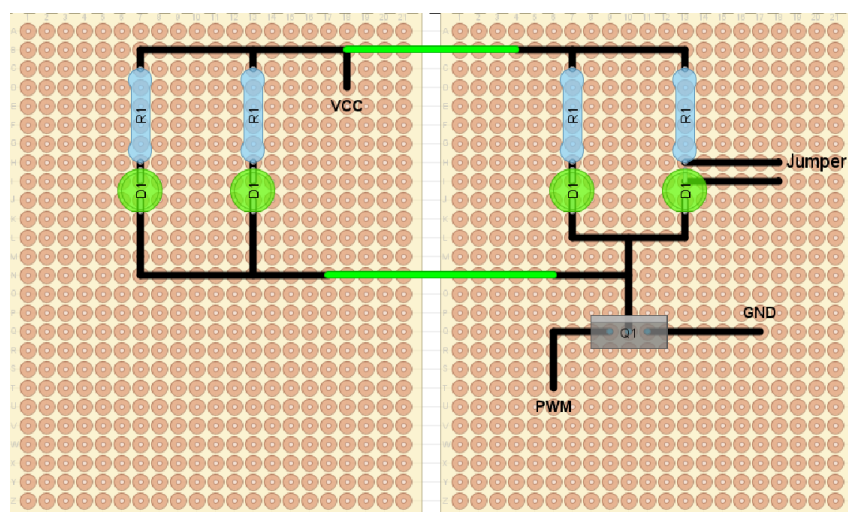


Figur 17: Spændingsfald for hvid LED (LTW-2S3D8)

Her indsættes beregninger, samt Multisim diagram



Figur 18 MultiSim diagram for forlyset. Bemærk at LED1/LED2 udgør det ene sæt forlys, og LED3/LED4 udgør det andet sæt.



Figur 19 Veroboard diagram for forlys

Stykliste for forlyls

Antal	Symbol på diagram	Komponent	Fabrikant/ Type
4	D1	Hvid LED	LTW-2S3D8
4	R1	Modstand	68Ω
1	Q1	Mosfet	IRLZ44N
1		Jumper	

Tabel 4 stykliste til verobordet for forlyset

Baglys:

Beregninger og konstruktion af kredsløbet er nøjagtigt det samme for baglyset, med undtagelse af at modstandene på LEDerne er større da spændingsfaldet henover de røde LEDer er mindre. I databladet aflæser vi et spændingsfald over LEDen på 2.3V ved en strøm på 25mA og vi kan derfor udregne at hver modstands størrelse bør være 108Ω.

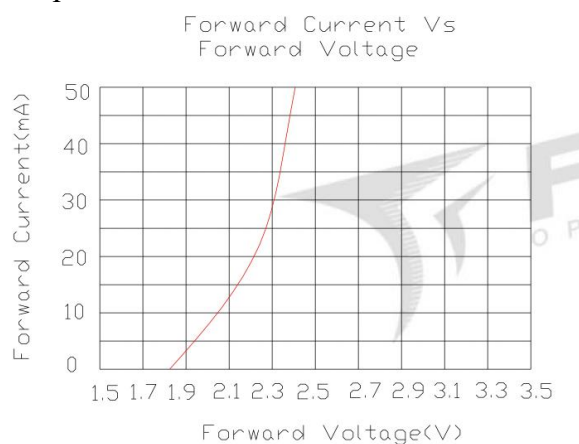
$$R_{Baglys} = \frac{V_{CC} - V_{LED, rød}}{25mA}$$

$$R_{Baglys} = \frac{5V - 2.3V}{25mA} = 108\Omega$$

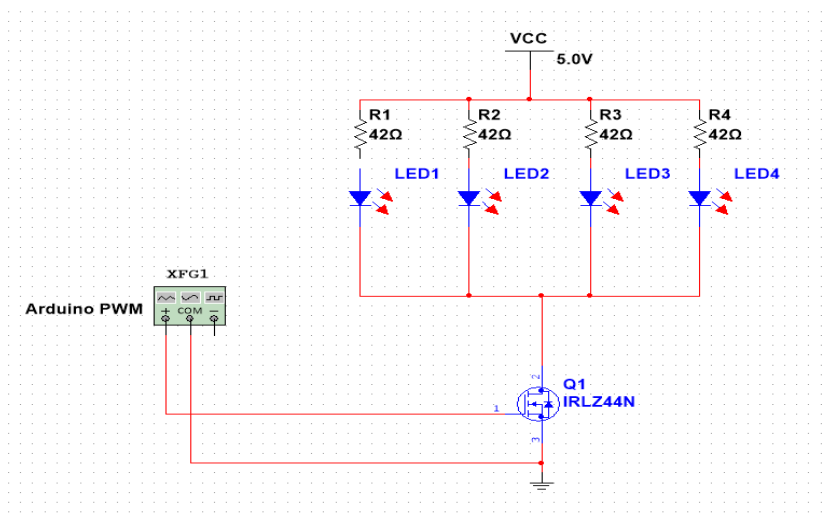
Ved konstruktion af kredsløbet på fumlebræt erfarer vi dog at en modstand på 108Ω er for stor til at vi kan måle 25mA igennem en enkelt LED. Dette forklares blot ved at komponenternes tolerancer. Matematikken fortæller os, at ved konstant spænding på 5V vil en modstand på 108Ω resultere i en strøm på 25mA igennem hver enkelt LED, men ved en duty-cycle på 100% måler vi 23mA. Det er naturligvis ikke langt fra hvad vi forventer, men vi vælger alligevel at nedjustere modstanden og bruger i stedet en på 42Ω. Det er tydeligvis en hel del lavere end den behøver at være, men det betyder at vi kan måle de korrekte strømme igennem LEDen ved køre- og bremselys på hhv. 5mA ved en duty-cycle på 11% og 25mA ved en duty-cycle på 55%.

Vi anvender samme mosfet (IRLZ44N) som for forlyls, og igen ignoreres spændingsfaldet herover da det er ubetydeligt småt.

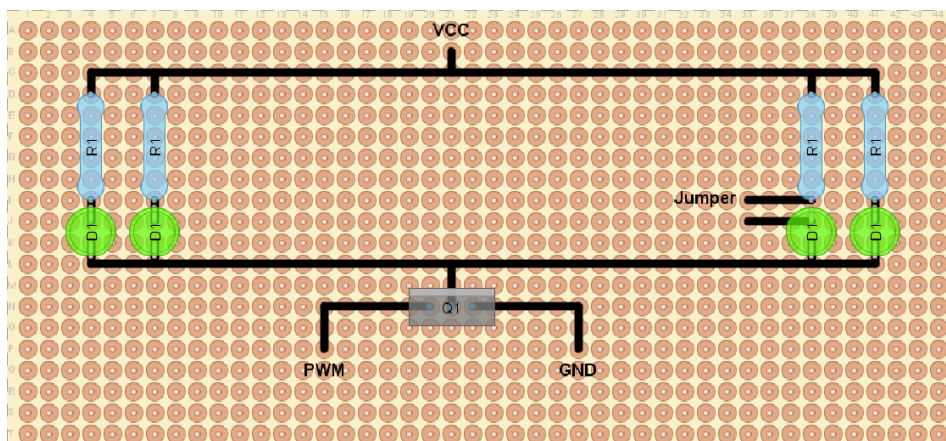
På vero-boardet er desuen monteret et par Harwin-pins med en jumper der tillader måling af strøm igennem én af LEDerne; dette er for at kunne bekræfte at kredsløbet opfylder vores kravspecifikation.



Figur 20: Spændingsfald for rød LED (FYL-5013HD1C)



Figur 21 Multisim diagram for baglyset



Figur 22 Veroboard diagram for baglyset

Stykliste for baglys

Antal	Symbol på diagram	Komponent	Fabrikant/ Type
4	D1	Rød LED	FYL-5013HD1C
4	R1	Modstand	42Ω
1	Q1	Mosfet	IRLZ44N
1		Jumper	

Tabel 5 stykliste til verobordet for baglyset

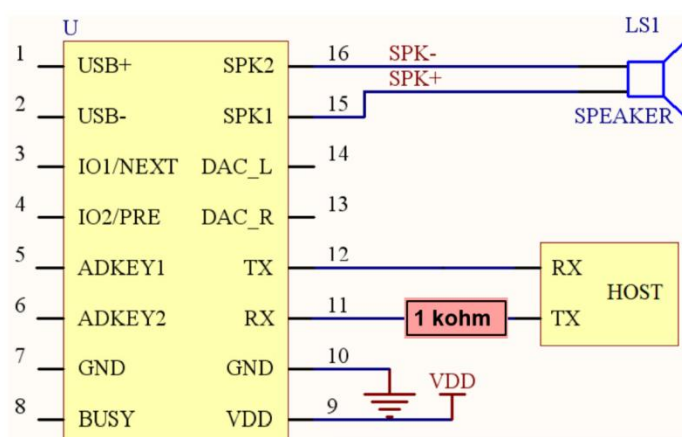
Lyd

Udarbejdet af Mikkel & Viktor

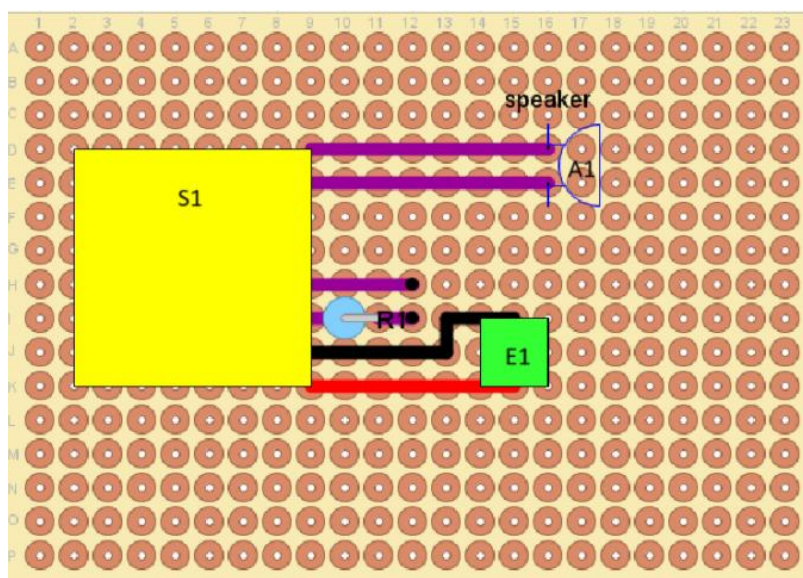
Formål

Lyden har til formål at afspille de passende lyde/sange til bilens tema. Lyden vil styres af en ARDUINO Mega 2560 som er tilkoblet en SOMO II der vil aflæse lydfileerne og til sidst vil det blive sendt ud igennem en højttaler. Den påkrævende strøm vil suppleres fra batteriet.

Diagrammer

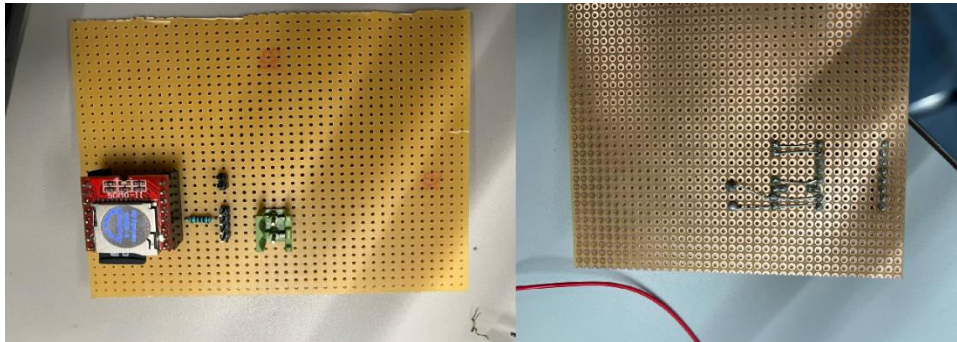


Figur 23. Print med øvrig elektronik placeret midt i bilen



Figur 24: Veroboard design af lyd-modulet

Ovenfor kan kredsløbet for at afspille lyden ses.



Figur 25: Billeder af lyd-veroboard (for og bag)

Stykliste

Antal	Komponent	Symbol	Fabrikant/type
1	Euroblok 2 stik	E1	
1	Modstand	R1	1 kohm
1	Højttaler	A1	PUI
1	SOMO II	S1	SOMO II
1	SD-kort		2 GB

Tabel 6 stykliste til verobordet for lyden

Driving Control

Udarbejdet af Oskar

Formål

Driving Control styrer bilens kørsel og behandler feedback fra reflekssensorerne. Til dette bruges reflekssensorerne, der er en ekstern enhed, som går HIGH, når en refleks detekteres. Dette outputsignal fra reflekssensorerne forbindes til henholdsvis den digitale pin 20 (INT1) og 21 (INT0). Disse to pins er hardware-interrupt pins og bruges til at reagere på eksterne signaler udenfor det normale loop. Denne forbindelse er vist på overbliksbilledet, se Figur 11. Samtidig modtager begge reflekssensorer 5V fra en af spændingsregulatorerne.

Med dette kredsløb er det muligt at udføre resten af signalbehandlingen i software. Dette er beskrevet under både softwaredesign og softwareimplementering.

6. Softwaredesign

Formålet med softwaredesign er at overskueliggøre softwareimplementeringen ved at beslutte hvad hver enkelt enhed skal kunne og hvordan det skal implementeres. Det forhindrer rod i koden og uhensigtsmæssige afhængigheder.

Motor

Udarbejdet af Andreas & Oskar

Formål

Motoren primære formål er at flytte bilen fra start til slut. For at gøre dette bruger vi tre funktioner, i overensstemmelse med vores softwarearkitektur, se Figur 10.

1. `initMotor()`
2. `setBackward(bool Backward)`
3. `setMotorIntensity(int intensity)`

Designvalg

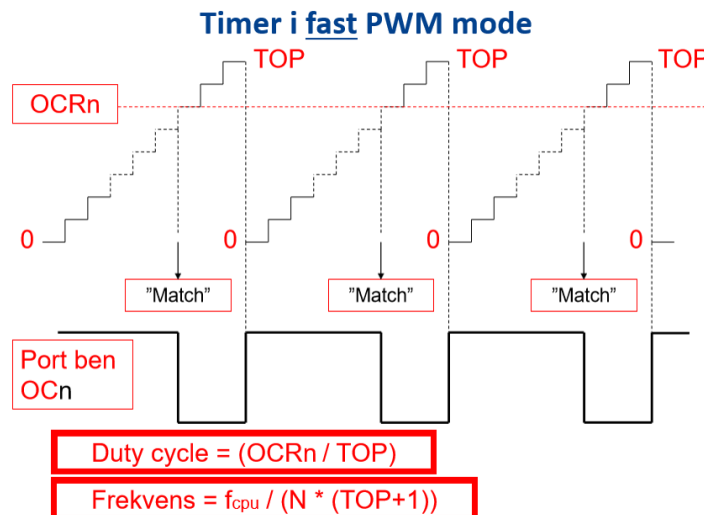
Først konfigureres motoren i `initMotor()`. A-systemet på Timer 1 er valgt, og svarer til den fysiske pin 11 på Arduinoen.

Timeren kører i Fast PWM, 10 bit og inverting mode, med en prescaler på 8, da det giver en frekvens på $\frac{16MHz}{8 \cdot (2^{10} + 1)} \approx 1951,22Hz$. Der er en passende hurtig frekvens til PWM, ikke for langsom til at motoren hakker, og samtidig ikke for hurtigt så tilstandsskift på mosfeten spilder for meget energi.

For at styre retningen af motoren laves en funktion, `setBackward(bool Backward)`, afhængig af, om Backward er True eller False, sætter Pin 23 enten HIGH eller LOW. Fordi Pin 23 er output, og Pin 23 er bit 1 på PORTA, konfigureres først Data Direction Registret (DDRA) til 1. Dermed sættes Pin 23 som output. Derefter sættes Pin 23 enten HIGH eller LOW.

For at styre motorens intensitet med A-systemet på Timer 1, der kører i PWM, 10 bit og inverting mode bruges `setMotorIntensity(int intensity)`.

For at forklare PWM signalet se Figur 26.



Figur 26. Viser hvordan en timer i fast PWM mode virker

Funktionen tager en parameter af typen integer, der bestemmer dutycyclen (0 - 100%). Som det fremgår af ovenstående figur, er output signalet HIGH, indtil der sker et match ved OCR1, da timeren er i inverting mode. Da der er 10 bit, er der $2^{10} = 1024$ steps. For at beregne OCR1A værdien bruges formlen:

$$OCR1A = \frac{intensity \cdot 1024}{100}$$

Med dette er det nu muligt at programmere motorens tre funktioner, se softwareimplementeringen.

Formål

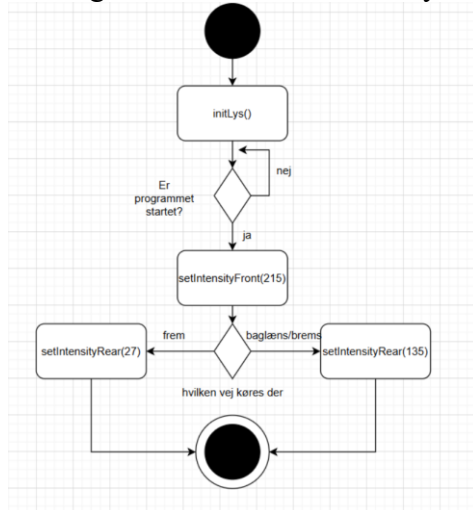
Software til lyset har det simple formål at regulere dutycycle i PWM-signalet der sendes til baglysene for at indikere køre- og bremselys. J.f. kravspecifikationen skal der ved kørellys måles 5mA igennem hver LED og ved bremselys 25mA som reguleres af Arduinoen. Lyset styres igennem 3 forskellige funktioner.

1. `initLys(Void);`
2. `setIntensityFront(int intensity);`
3. `setIntensityRear(int intensity);`

Designvalg

LED-driveren har til opgave at tænde lyset ved bilens opstart og slukke lyset efter bilen har gennemført kørslen af banen. Justering af lysstyrken styres -ved passering af specifikke refleksbrikker, styres i vores Driving Control.

For Arduinoen er Timer 0 er anvendt til både for og baglys. PG5, som svarer til Pin 4 på ATmega 2560 er anvendt til forlys og PB7, som svarer til Pin 13 er anvendt til baglys.



Figur 27: UML activity diagram for lys

Driving Control styrer, hvornår lyset tændes og slukkes. Lyset bliver styret med tre funktioner: `initLys()`, `setIntensityFront(int intensity)` og `setIntensityRear(int intensity)`.

Når systemet starter, kaldes `initLys()` for at gøre lysene klar. Derefter tændes forlygterne med en fast styrke (`setIntensityFront(215)`).

Alt efter om bilen kører frem eller baglæns, justeres baglygterne:

- Hvis bilen kører frem, lyser baglygterne svagt (`setIntensityRear(27)`).
- Hvis bilen bremser eller bakker, lyser baglygterne kraftigere (`setIntensityRear(135)`).

Dette design er valgt, fordi det er enkelt og nemt at arbejde med, og det er let at ændre eller bygge videre på, hvis der skal laves mere senere.

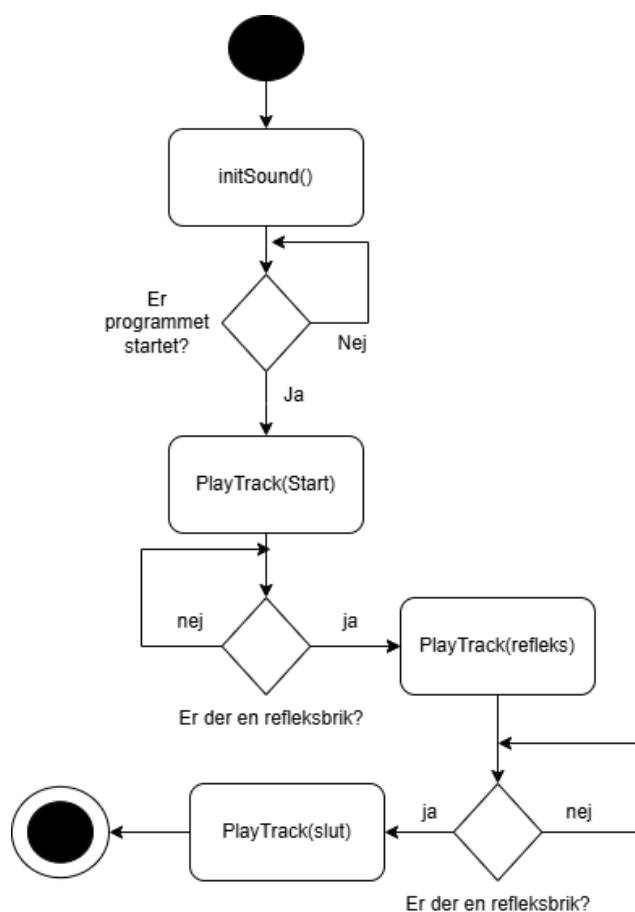
Formål

Lydens software har til formål, at kunne afspille lyd ved 3 forskellige instanser: Ved bilens start, efter passering af en refleksbrik og ved afslutning af bilens kørsel. For at gøre dette, vil der blive anvendt følgende funktioner:

1. InitSound();
2. PlaySound(Int sound);

Designvalg

Vores sound driver er afhængig af få ting: Om programmet er tændt, hvor mange refleksbrikker er ramt og er bilen i mål. En specifik lyd vælges herefter passende til den respektive use case. Se gerne figur 28



Figur 28. UML activity digram for lyd

Det er Driving Control der styrer hvornår lyden afspilles og dette sker i funktionen drive(), hvor både motor, lys og lyd styres synkront fra refleksdataen.

Sound Driver er struktureret som en selvstændig driver, der har funktionerne PlaySound(Int sound) og InitSound(). Disse vil sikre, at lyden bliver afspillet og at lyden altid er skruet helt op.

Lyden styres af to funktioner, `PlaySound(int sound)` og `initSound()`. `initSound` vil blive brugt til at initiere lyden, hvilket er at initiere uart og lydens lydstyrke. Herefter bruges `PlaySound` til at bestemme hvilken lyd der skal spilles. Lydfunktionen anvendes i forbindelse med systemstart og -slut, samt passering af en refleksbrik. Ved opstart afspilles en startlyd (`PlaySound(start)`), og ved afslutning en slutlyd (`PlaySound(slut)`). Mellem disse lydpunkter afventer systemet en detektion af refleks, og først når en refleksbrik er registreret (via funktionen `detectRefleks`), afspilles den relevante feedbacklyd (`PlaySound(refleks)`). Denne sekvens fremgår af aktivitetsdiagrammet nedenfor.

Som det fremgår, initialiseres lyden ved opstart, hvorefter en startlyd afspilles. Systemet går derefter i en ventetilstand, hvor det afventer, at en refleksbrik bliver ramt. Når dette sker, udløses en ny lyd, og efter en afsluttende vurdering af de antal use cases afspilles en slutlyd når det rigtige antal refleksbrikker er ramt. Dermed fuldendes den auditive feedbacksekvens i samspil med refleksdetektion og kørekontrol.

Driving Control

Udarbejdet af Oskar

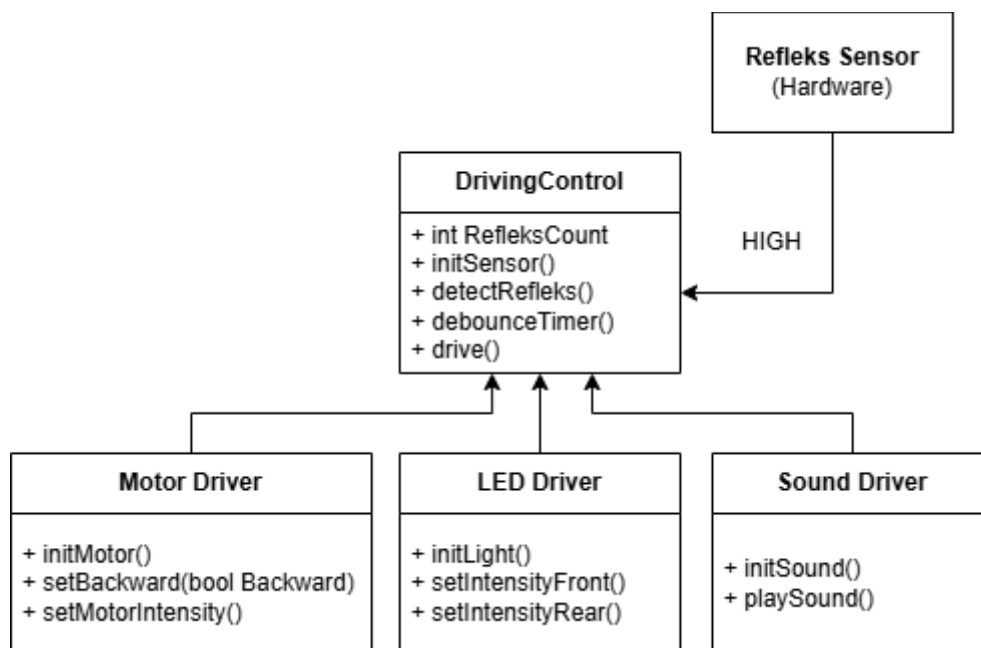
Formål

Driving Control har to formål, primært at styre bilens kørsel, og sekundært at behandle feedback fra reflekssensorene, dog afhænger bilens kørsel af denne funktionalitet. For at gøre dette bruger vi 4 funktioner, i overensstemmelse med vores softwarearkitektur, se Figur 10.

1. `initSensor()`
2. `detectRefleks()`
3. `debounceTimer()`
4. `drive()`

Designvalg

Vigtigst er det, at Driving Control kan styre bilens kørsel baseret på placeringen på banen. Placeringen vurderes ved at registrere refleksbrikker med en reflekssensor, der er en selvstændig hardwareenhed, på hver side af bilen. Derefter justeres motorens hastighed, lysets intensitet og lydfilen samt lydstyrken i overensstemmelse med placeringen på banen. Dette sker i en funktion kaldet 'Drive' i overensstemmelse med vores softwarearkitektur. Forholdet mellem bilens drivere er vist på nedenstående UML Diagram.



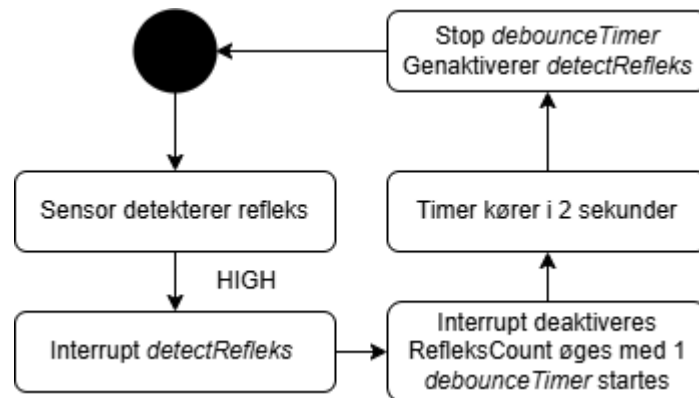
Figur 29. Viser forholdet mellem DrivingControl, reflekssensorerne og de andre drivere.

Som det fremgår af den ovenstående figur, modtager Driving Control et HIGH fra reflekssensoren, idet der registreres en refleks. Driving Control afhænger af denne feedback, men kan eksistere foruden, selvom det går udover funktionaliteten. Samtidig er der en association mellem Driving Control og de tre drivere: 'Motor Driver', 'LED Driver' og 'Sound Driver', idet Driving Control bruger eksistensen af de 3 drivere og kalder deres funktioner i funktionen 'Drive'.

Funktionen 'Drive' er i den grad implementeret kvalitativt, forstået på den måde, at den switcher på størrelsen af RefleksCount, og til hver switchcase er motorens intensitet, lysets

intensitet, lydfilen og lydstyrken vurderet baseret på tests på banen. Den endelige funktion fremgår af softwareimplementeringen.

Den anden funktion af Driving Control er at behandle feedbacken fra reflekssensorerne. Logikken bag behandlingen af feedback fra reflekssensorerne gennemgås her, og selve kode implementeringen er vist under softwareimplementeringen.



Figur 30. Aktivitets UML-diagram, der viser hvordan feedbacken fra reflekssensorerne foregår.

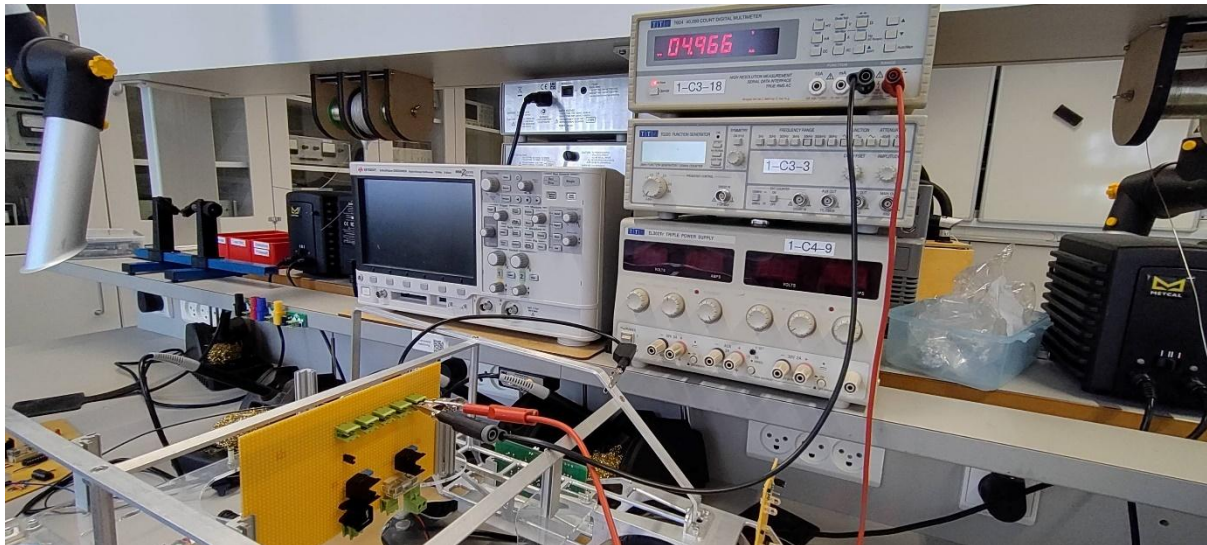
Som det fremgår af ovenstående figur, går reflekssensorerne på hver side af bilen HIGH når der detekteres en refleksbrik. Denne feedback fra sensoren, aktiverer et interrupt, der kalder en funktion kaldet `detectRefleks`, som inkrementerer en variabel, der holder styr på antallet af reflekser kaldet 'RefleksCount'. Samtidig deaktiveres selvsamme interrupt, og en timer startes kaldet `debounceTimer`. Efter timeren går i overflow (1 sekund) genaktiveres interruptet `detectRefleks` og timeren stoppes. Dermed er bilen klar til at detektere en ny refleks.

7. Hardwareimplementering og modultest

Formålet med Hardwareimplementering og modultest er at eftervise at hardwaremodulerne på basis af HW-arkitekturen og HW-designet fungerer efter hensigten og i overensstemmelse med kravene. Hvert afsnit vil have fokus på krav, modultests samt resultater.

Spændingsregulator

Udarbejdet af Andreas & Oskar



Figur 31: Test af spændingsregulator ved brug af multimeter.

Testen bruger samme navne for terminalerne som vist på Figur 13.

E1 tilkobles 9,6V Batteri, derefter måles spændingen over alle terminaler

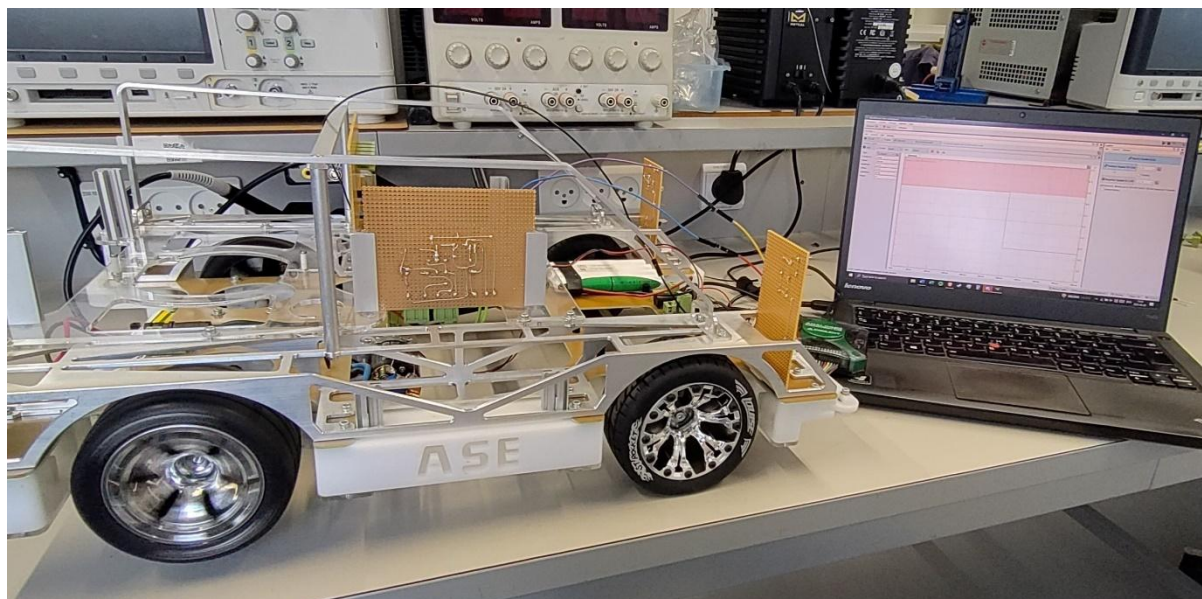
E1	E2	E3	E4	E5	E6	E7	E8
10,13 V	10,08 V	4,96 V	4,97 V	4,96 V	4,97 V	4,97 V	4,97 V

Tabel 7. Resultater for test af spændingsregulator

E1 og E2 spænding er højere end forventet, dette skyldes et overopladt batteri.
E3-E8 har den ønskede spænding, tæt på 5 V.

Motorstyring

Udarbejdet af Andreas & Oskar



Figur 32. Test af moter ved brug af Analog Discovery.

Motoren tilkobles Analog Discovery. PWM-signalet styres af wavegen og relæspolen af et 5V signal fra supplies i Analog Discovery.

Wavegen sættes til at generere en square-wave på 1951,22 Hz, derefter kan dutycycle varieres. Supply signalet ændres mellem 5 V eller 0 V.

PWM, Dutycycle	Relæspole, supply 0V	Relæspole, supply 5V
0%	Ingen bevægelse	Ingen bevægelse
25%	Forlæns kørsel	Baglæns kørsel
50%	Hurtig forlæns kørsel	Hurtig baglæns kørsel
75%	Hurtigere forlæns kørsel	Hurtigere baglæns kørsel
100%	Hurtigste forlæns kørsel	Hurtigste baglæns kørsel

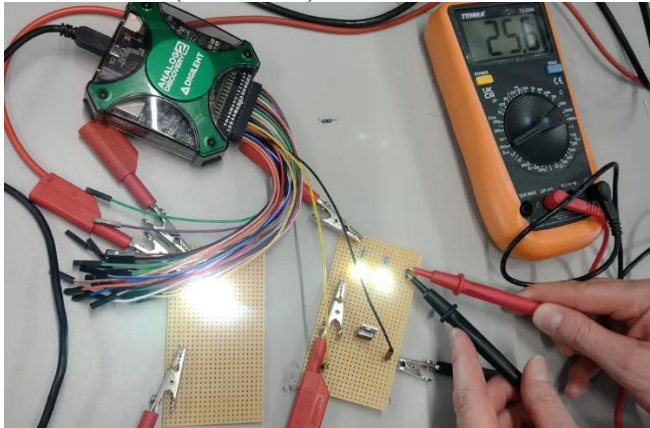
Tabel 8. Resultater for test af moter

Motoren fungerer som den skal. Hastigheden og retningen af kørsel kan styres med PWM og signalet til Relæspolen.

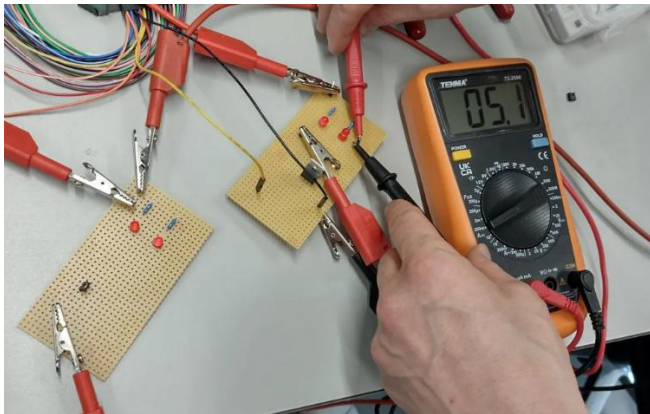
Lys

Udarbejdet af Morten, Tim & Frederik

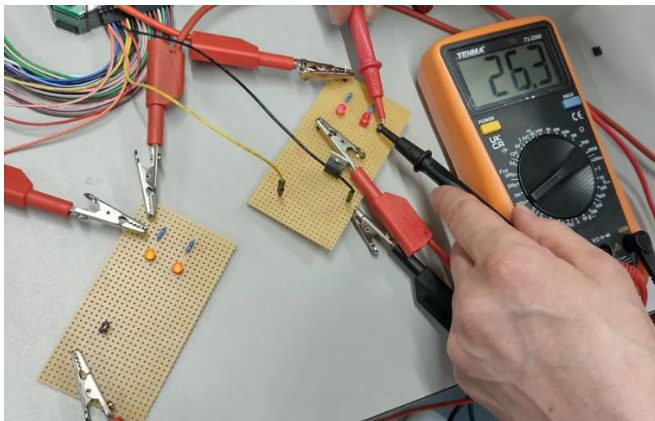
Lyset tilkobles spændingsforsyningen i laboratoriet og Analog Discovery. PWM-signalet styres af wavegen hvor vi ved en trial-and-error proces afprøver adskillige duty-cycle værdier indtil vi måler de rette mA i kredsløbet (i overensstemmelse med værdierne i vores kravspecifikation). Vi bekræfter i disse forsøg at kredsløbene virker som de skal og at især baglysets intensitet kan kontrolleres ved at variere duty-cycle på PWM signalet der modtages af mosfetten (IRLZ44N).



Figur 33: Test af lys. Under test af forlyset målte vi 25mA ved en duty cycle på 87%.



Figur 34: test af lys. Under test af baglyset (kørelys) målte vi 5mA ved en duty cycle på 11%.



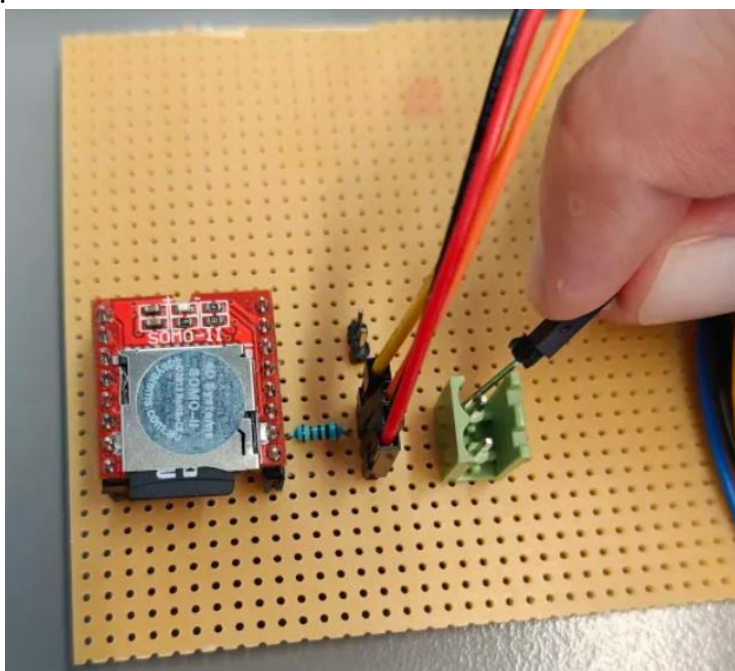
Figur 35: Test af baglys. Under test af baglyset (bremselys) målte vi 26mA ved en duty cycle på 57%.

Testopsætning

Lydsystemet blev tilsluttet 5V fra spændingsregulatoren og styret via Arduino Mega2560. Arduinoen kommunikerer med SOMO II-lydmodulet, som afspiller MP3-filer fra et 2GB microSD-kort. Lydsignalet går videre til højttaleren, der er tilsluttet SOMO II-lydmodulet

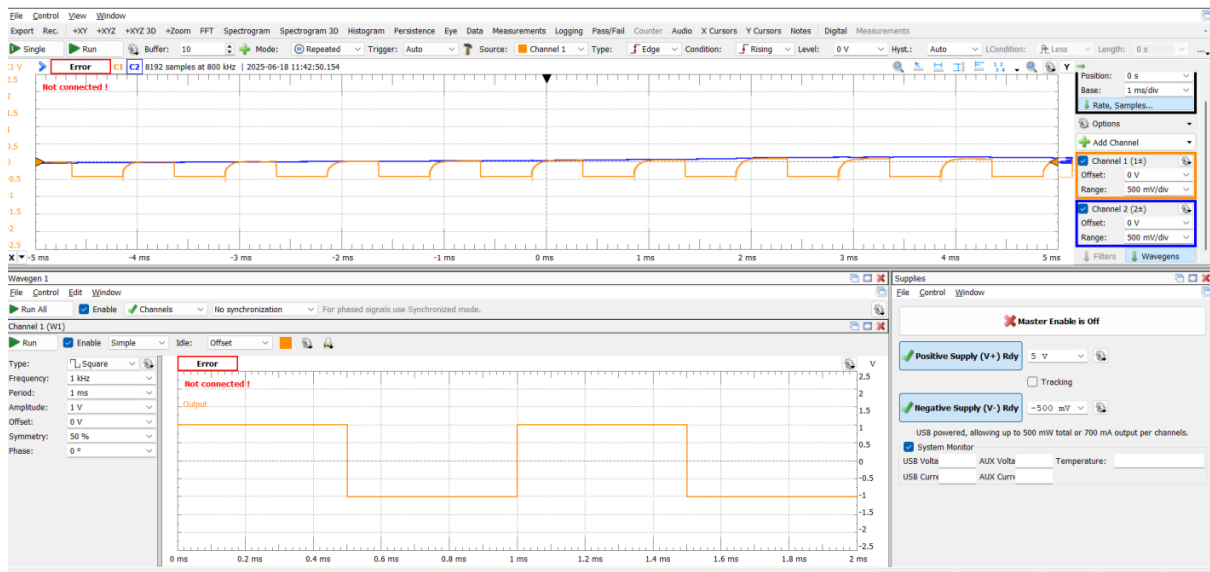
Testforløb

Testen blev udført ved at aktivere hver af de tre lydscenarier individuelt ved brug af testfunktioner i Arduino-koden, der simulerer hændelserne (start, refleks, slut). Til udførslen af denne test blev der anvendt MEGA 2560 I/O shield hvori hver "knap" på shield'et blev sat til testning af en lydfil. Et dB-meter blev brugt til at måle lydniveauet ved 2 meters afstand. Før arduino bliver testet, skal selve SOMO'en testes. Dette gøres ved at sende et signal gennem dens receiver, herefter tager man signalet fra den transmitter, da SOMO'en selv udgiver et signal for hver gang den modtager et. En analog discovery bliver brugt til at sende strøm, samt et signal gennem SOMO'en. Samt bruges den også til at fange signalet som SOMO'en sender.



Figur 36:

Herefter bruges der waveforms, til at se signalet og om ens opbygning virker.



Figur 37

I øvre halvdel, ved det orange signal, kan der ses at den sender et signal ud. I nedre halvdel kan det ses at der bliver sendt strøm til SOMO'en, samt at der bliver sendt et signal til den. Herefter kan der så bruges en testkode, for at se om højttaleren og SOMO'en virker sammen, ved test af switches.

Koden der udregner checksum. Checksum skal beregnes for bit 5 og 6. Checksum er forskellig for hver kommando der sendes, og skal derfor beregnes ved hvert brug.	<pre>void calculateChecksum(unsigned char cmd[], unsigned char* high, unsigned char* low) { unsigned int sum = cmd[1] + cmd[2] + cmd[3] + cmd[4]; unsigned int checksum = 0xFFFF - sum + 1; *high = (checksum >> 8) & 0xFF; *low = checksum & 0xFF; }</pre>
Koden der sender en funktion afsted	<pre>void sendSomoCommand(unsigned char cmd[8]) { for (uint8_t i = 0; i < 8; i++) { SendChar(cmd[i]); } }</pre>
Koden der tester beregning af checksum, herefter koden der sender funktionen afsted. volumen sættes til 30, hvilket er max	<pre>int main(void) { InitUART(9600, 8, 0); initSwitchPort(); initLEDport(); _delay_ms(3000); // Boot time unsigned char checksumHigh, checksumLow; // Set volume unsigned char setVolume[8] = {0x7E, 0x06, 0x00, 0x00, 0x1E, 0x00, 0x00, 0xEF}; calculateChecksum(setVolume, &checksumHigh, &checksumLow); setVolume[5] = checksumHigh; setVolume[6] = checksumLow; sendSomoCommand(setVolume); _delay_ms(500); }</pre>

I while-loopet skrives kode, til at hvis man trykker på knappen tilhørende en switch, så vil switchen tænde, tænde en LED, samt sende en lyd afsted, der er forudbestemt.

```
while (1) {
    if (switchOn(0)) {
        turnOnLED(0);

        unsigned char playTrack[8] = {0x7E, 0x03, 0x00, 0x00, 0x01, 0x00, 0x00, 0xEF};
        calculateChecksum(playTrack, &checksumHigh, &checksumLow);
        playTrack[5] = checksumHigh;
        playTrack[6] = checksumLow;
        sendSomoCommand(playTrack);

        while (switchOn(0)) {}
        _delay_ms(200);
        turnOffLED(0);
    }

    if (switchOn(1)) {
        turnOnLED(1);

        unsigned char playTrack[8] = {0x7E, 0x03, 0x00, 0x00, 0x02, 0x00, 0x00, 0xEF};
        calculateChecksum(playTrack, &checksumHigh, &checksumLow);
        playTrack[5] = checksumHigh;
        playTrack[6] = checksumLow;
        sendSomoCommand(playTrack);
        while (switchOn(1)) {}
        _delay_ms(200);
        turnOffLED(1);
    }

    if (switchOn(2)) {
        turnOnLED(2);

        unsigned char playTrack[8] = {0x7E, 0x03, 0x00, 0x00, 0x03, 0x00, 0x00, 0xEF};
        calculateChecksum(playTrack, &checksumHigh, &checksumLow);
        playTrack[5] = checksumHigh;
        playTrack[6] = checksumLow;
        sendSomoCommand(playTrack);

        while (switchOn(2)) {}
        _delay_ms(200);
        turnOffLED(2);
    }
}
}
```

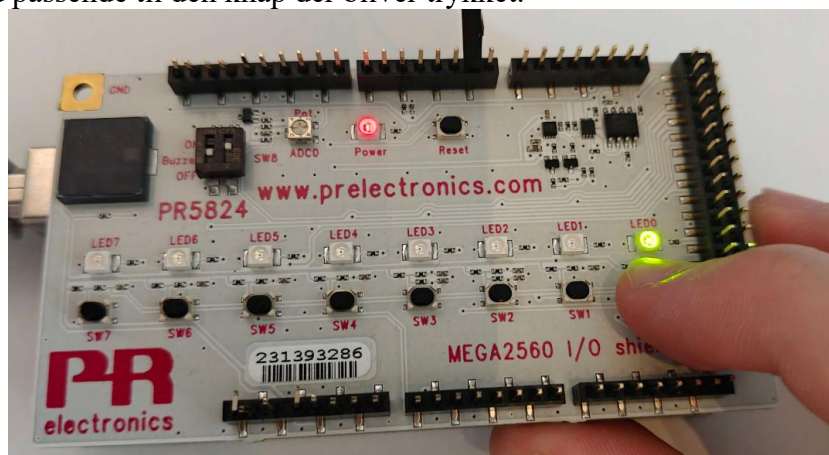
Koden der gør det muligt at bruge switchen til resten af sin kode.

```
void initSwitchPort() {
    DDRA &= ~(1 << PA0); // Set PA0 as input
    PORTA |= (1 << PA0); // Enable pull-up resistor on PA0
}

unsigned char switchStatus() {
    return ~PINA; // Invert because pull-ups make pressed == 0
}

unsigned char switchOn(unsigned char switch_nr) {
    if (switch_nr > 7) return 0;
    return !(PINA & (1 << switch_nr)); // Active low logic
}
```

Herefter for at teste det, skal det sættes ned på en Arduino med et I/O shield på. Koden vil så afspille en lyd passende til den knap der bliver trykket.



Figur 38

Knappen trykkes og der tændes en LED. Herefter spilles lyden ud fra SOMO'en.

8. Softwareimplementering og modultest

Formålet med Softwareimplementeringen og modultest er at eftervise at softwaremodulerne på basis af SW-arkitekturen og SW-designet fungerer efter hensigten og i overensstemmelse med kravene.

Motor

Udarbejdet af Andreas & Oskar

Krav

Skal kunne styre motorens retning og intensitet ved brug af et PWM-signal genereret på Arduinoen.

Implementering

Sætter PB5, der er den fysiske Pin 11 på Arduinoen som output, da det er outputtet på system A på Timer 1. Derefter sættes timeren til Fast PWM, 10 bit, inverting mode med en prescaler på 8. Det giver en frekvens på 1953,125 Hz.	<pre>void initMotor() { DDRB = (1 << DDB5); TCCR1A = (1 << COM1A1) (1 << WGM11) (1 << WGM10); TCCR1B = (1 << WGM12) (1 << CS11); };</pre>
Sætter Pin 23 som output Hvis bilen skal køre baglæns, går Pin 23 HIGH. Hvis bilen ikke skal køre baglæns, altså forlæns, går Pin 23 LOW.	<pre>void setBackward(bool Backward) { #define PIN_23 1 DDRA = (1 << PIN_23); if (Backward) { PORTA = (1 << PIN_23); } else { PORTA &= ~(1 << PIN_23); } };</pre>
Løser problemet hvis intensiteten går over 100 eller under 0. Beregner den nødvendige OCR1A værdi, sådan at PWM signalet får den korrekte dutycycle.	<pre>void setMotorIntensity(int intensity) { if (intensity < 0) intensity = 0; if (intensity > 100) intensity = 100; OCR1A = (intensity * 1023UL) / 100; };</pre>

Modultest

Motorens software er testet på forskellige cases, der tager højde for alle de forskellige scenarier.

Test	Åbnet Resultat
initMotor() setBackward(1) setMotorIntensity(120)	Motoren kører baglæns med 100% intensitet
initMotor() setBackward(0) setMotorIntensity(50)	Motoren kører forlæns med 50% intensitet
initMotor() setBackward(0) setMotorIntensity(0)	Motoren står stille.

Tabel 9. Resultater for test af motor

Lys

Udarbejdet af Morten, Frederik & Tim

Krav

Skal kunne styre for og baglyset intensitet via PWM gennem Arduino ATmega2560.

Implementering

De forskellige funktioner i LED driveren er beskrevet nedenfor.

Sætter PG5 Pin 4 som output til forlys. Timer0 anvendes for hele systemet OCROB til forlys. OCROA til baglys Sætter PB7 Pin 13 som output til baglys. PORTG slukker forlyset ved at sætte PG5 LOW. PORTB slukker for baglyset ved at sætte PB7 LOW.	<pre>void initLys(void) { // Forlys DDRG = (1 << PG5); TCCR0A = (1 << COM0B1) (1 << WGM01) (1 << WGM00); TCCR0B = (1 << CS00); OCR0B = 0; // Baglys DDRB = (1 << PB7); TCCR0A = (1 << COM0A1); OCR0A = 0; PORTG &= ~(1 << PG5); PORTB &= ~(1 << PB7); }</pre>
Funktion som styrer forlys intensitet. Lys intensitet styres Sender PWM værdien til OCROB Sikrer en PWM værdi mellem 0 - 255	<pre>void setIntensityFront(int intensity) { if (intensity < 0) intensity = 0; if (intensity > 255) intensity = 255; OCR0B = intensity; }</pre>
Funktion som styrer baglys intensitet. Lys intensitet styres Sender PWM værdien til OCROA Sikrer en PWM værdi mellem 0 - 255	<pre>void setIntensityRear(int intensity) { if (intensity < 0) intensity = 0; if (intensity > 255) intensity = 255; OCR0A = intensity; }</pre>

Modultest

Modultesten går ud på at tjekke, at initLys() starter lyset korrekt, og at setIntensityFront(intensity) og setIntensityRear(intensity) kan tænde og slukke forlygter og baglygter med den rigtige styrke, alt efter hvilken værdi de får. Hvis lysene reagerer rigtigt, er testen godkendt.

Test	Åbnet Resultat
initLys() setIntensityFront(215) setIntensityRear(27)	Forlys lyser kraftigt. Baglys lyser svagt
initLys() setIntensityFront(215) setIntensityRear(135)	Forlys lyser kraftigt. Baglys lyser kraftigt
initLys() setIntensityFront(0) setIntensityRear(0)	Begge lys er slukket

Tabel 10. Resultater for test af lys

Lyd

Udarbejdet af Mikkel og Viktor

Krav

Bilen skal afspille en start, refleksbrik og -slutlyd på de passende tidspunkter

Implementering

Lyden initialiseres.
Volumen bliver sat til maksimum og kommandoen sendes til Arduinoen, som herefter sender det videre til SOMO'en.

```
void initSound(void) {  
    InitUART(9600, 8, 0); // Initialize UART at 9600 baud rate  
    _delay_ms(3000); // Wait for boot-up  
  
    // Set volume  
    unsigned char setVolume[8] = {0x7E, 0x06, 0x00, 0x00, 0x1E, 0x00, 0x00, 0xEF};  
  
    // Checksum calculation for volume command  
    unsigned int sum = setVolume[1] + setVolume[2] + setVolume[3] + setVolume[4];  
    checksum = 0xFFFF - sum + 1;  
    checksumHigh = (checksum >> 8) & 0xFF;  
    checksumLow = checksum & 0xFF;  
    setVolume[5] = checksumHigh;  
    setVolume[6] = checksumLow;  
  
    // Send volume command  
    for (uint8_t i = 0; i < 8; i++) {  
        SendChar(setVolume[i]);  
    }  
    _delay_ms(500); // Wait for the command to take effect  
}
```

En bestemt lyd bliver spillet ud for hvad der er valgt.
1 er startlyden.
2 er reflekslyden.
3 er slutlyden.

Herefter sendes det til Arduinoen, som herefter sender det videre til SOMO'en.

```
void PlaySound(int sound) {  
    if (sound == 1) { // Assuming '1' corresponds to start  
        playTrack[4] = 0x01; // Set start sound  
    }  
    else if (sound == 2) { // Assuming '2' corresponds to refleks  
        playTrack[4] = 0x02; // Set refleks sound  
    }  
    else if (sound == 3) { // Assuming '3' corresponds to slut  
        playTrack[4] = 0x03; // Set slut sound  
    }  
  
    // Calculate checksum for the playTrack array  
    unsigned int sum = playTrack[1] + playTrack[2] + playTrack[3] + playTrack[4];  
    checksum = 0xFFFF - sum + 1;  
    checksumHigh = (checksum >> 8) & 0xFF;  
    checksumLow = checksum & 0xFF;  
    playTrack[5] = checksumHigh;  
    playTrack[6] = checksumLow;  
  
    // Send playTrack command  
    for (uint8_t i = 0; i < 8; i++) {  
        SendChar(playTrack[i]);  
    }  
}
```

Modultest

For at lave en modultest for lyden, gøres der brug af koden. Dette sættes i en main funktion, hvorefter man bruger et while-loop til at spille hver lyd, og starte forfra. Det tilkobles til en arduino og en SOMO.

Her bruges en main funktion, hvorefter lyden initialiseres. Derefter spiller man lydene igen og igen i et while-loop.

```
int main(void) {  
    initSound();  
    while (1){  
        PlaySound(1);  
        _delay_ms(5000);  
        PlaySound(2);  
        _delay_ms(5000);  
        PlaySound(3);  
        _delay_ms(5000);  
    }  
}
```

Driving Control

Udarbejdet af Oskar og Andreas

Krav

Registrere refleksbrikker og derigennem styre bilens kørsel.

Implementering

De forskellige funktioner i Driving Control er beskrevet nedenfor, deres indbyrdes forhold er beskrevet i softwaredesignet. Den endelig Drive funktion er tilpasset igennem tests på banen, og benytter sig af alle de forskellige drivere, og kan findes som bilag i sourcekoden.

Sættes til at trigge på falling edge, og aktiverer de to hardware interrupt pins på INT0 og INT1.	<pre>void initSensor() { EICRA = (1 << ISC01); EICRA &= ~(1 << ISC00); EIMSK = (1 << INT0); EIMSK = (1 << INT1); }</pre>
Først deaktiveres de to eksterne interrupts INT0 og INT1. RefleksCount inkrementeres med 1 Debounce timeren aktiveres Drive funktionen, aktiveres kun idet en ny refleks detekteres.	<pre>void detectRefleks(){ EIMSK &= ~(1 << INT0); EIMSK &= ~(1 << INT1); RefleksCount++; debounceTimer(); drive(); }</pre>
Sætter timeren til CTC mode og en prescaler på 1024. Skaber et compare match efter 1 sekund, ved at sætte OCR3A = 15625. Da timer 3 er en 16-bit timer, og kører med en frekvens på $\frac{16MHz}{1024} = 15625Hz$. Når den derfor 15625 tællinger efter 1 sekund. Aktiverer A systemet på timer 3	<pre>void debounceTimer () { TCCR3B = (1 << WGM32); TCCR3B = (1 << CS32) (1 << CS30); OCR3A = 15625; TCNT3 = 0; TIMSK3 = (1 << OCIE3A); }</pre>

Drive funktionen bliver kaldt hver gang refleksbrik bliver detekteret og når bilen startes. Værdien i RefleksCount bruges i en Switch-case til at indstille bilen ved brug af den andre funktioner. Funktionen har indstillinger op til case 11, hvorefter at bilen vil have gennemført banen	<pre>void drive(){ switch(RefleksCount){ case 0: PlaySound(1); _delay_ms(4000); setBackward(0); setMotorIntensity(50); setIntensityFront(215); setIntensityRear(27); break; case 1: PlaySound(2); break; case 2: PlaySound(2); setMotorIntensity(100); break; case 3: PlaySound(2); setMotorIntensity(20); break; case 4:</pre>
Hvis de ikke er deaktiveret, vil INT0 og INT1 køre detectRefleks funktionen når der detekteres en refleks i den ene eller anden side. INT0 og INT1 bliver deaktiveret i detectRefleks funktionen og TIMER3 bliver tændt Når TIMER3 når til compare-værdien bliver timeren slukket, og INT0 og INT1 bliver genaktiveret	<pre>ISR(INT0_vect) { detectRefleks(); } ISR(INT1_vect) { detectRefleks(); } ISR(TIMER3_COMPA_vect) { TCCR3B = 0; TIMSK3 &= ~(1 << OCIE3A); EIFR = (1 << INTF0); EIFR = (1 << INTF1); EIMSK = (1 << INT0); EIMSK = (1 << INT1); }</pre>

Modultest

Reflekssensoren skal udelukkende kunne opfange en refleksbrik hvert sekund. Denne funktionalitet er testet ved brug af Analog Discovery. Der er målt på output Pin 23, der bruges til at styre retningen af motoren, og tidsintervallet mellem HIGH og LOW er herigennem målt til at være 1 sekund. Dette er i overensstemmelse med funktionaliteten af debounceTimeren.

Drive funktionen testes ved at starte bilen, hvorefter en refleks bliver ført foran reflekssensoren. Ved at lytte til bilens lyde og se hvordan motoren kører, kan vi se at bilen følger den korrekte sekvens af indstillinger hver gang en refleks bliver ført foran reflekssensoren. Hvis refleksen bliver først for hurtig foran reflekssensoren bliver det ignoreret.

9. Integrationstest

Udarbejdet af alle i gruppen

Efter at have færdigbygget modulerne, kunne vi montere dem på bilen ved at sætte dem i de designede holdere. Takket være grundig forberedelse gik integreringen af komponenterne problemfrit. For at sikre pålidelighed under kørslen, specielt på grund af de rystelser som konkurrencerampen ville forårsage, fastgjorde vi flere ledningsforbindelser til bilens stel for at forhindre hardwarefejl.

Softwareændringerne var minimale og omfattede justering af bilens hastighed samt finjustering af debounce-timeren for at sikre pålidelig detektion af alle reflekser.

Med alle komponenter monteret og funktionsdygtige på bilen kunne vi gennemføre accept-testene og evaluere, om det endelige system opfyldte de specificerede krav.



Figur 39. Billede af bilen med alle moduler installeret, på nær skal til tema.



Figur 40. Billede af bilen med tema.

10. Accepttestspecifikation

Udarbejdet af hele gruppen

Indledning

Denne accepttestspecifikation beskriver de testmetoder og kriterier, der anvendes til at verificere, at bilen opfylder de specificerede krav. Accepttesten gennemføres til sidst for at sikre, at bilens funktionelle og ikke-funktionelle krav er korrekt implementeret og opfylder forventningerne beskrevet i kravspecifikationen.

Testene er baseret på bilens use cases og tekniske specifikationer, og hver test indeholder en beskrivelse af testscenariet, forventet resultat samt opnået resultat. Accepttesten udføres af projektgruppen for at validere systemets funktionalitet før endelig godkendelse.

For at kunne optimere udarbejdelsen af produktet, så er kravene blevet vurderet via en MoSCoW. En MoSCoW kravspecifikation er en metode, der bruges til at prioritere krav og funktioner i et projekt. Navnet **MoSCoW** er et akronym, hvor hvert bogstav står for en kategori af krav:

- **M – Must have** (skal have): Disse krav er essentielle for projektets succes og skal være opfyldt, ellers vil projektet ikke blive anerkendt som færdigt eller funktionelt.
- **S – Should have** (bør have): Disse krav er vigtige, men ikke nødvendige for projektet. De bør inkluderes, hvis der er tid og ressourcer, men kan udskydes, hvis der er andre prioriteringer.
- **C – Could have** (kunne have): Disse krav er ønskelige, men ikke kritiske. De er typisk "nice-to-have" funktioner, som kun bør implementeres, hvis der er tid til overs.
- **W – Won't have** (vil ikke have): Dette er krav, der ikke er nødvendige for den aktuelle version af projektet. De kan blive overvejet til fremtidige versioner eller opdateringer.

MoSCoW-metoden hjælper med at skabe en klar prioritering og sikrer, at de vigtigste funktioner bliver implementeret først, samtidig med at man undgår at bruge ressourcer på mindre vigtige funktioner.

Kravene er vurderet ud fra, at "Bilen skal kunne deltage i konkurrencen"

11.2. Accepttestspecifikation for funktionelle krav (Use Cases)

Use Case 1: "Kør banen"	Test	Forventet resultat	Opnået resultat	Godkendt/ kommentar
Punkt 1 + punkt 2	Brugeren tænder for bilen og placerer den, så den kører forlæns ind på banen ved at passere startlinjen. Når bilen har passeret refleksbrik nummer 6, bringes bilen til standsning inden refleksbrik nummer 7 nås.	Visuel test: Bilen standser mellem refleksbrikkerne 6 og 7.	Bilen standser mellem refleksbrikkerne 6 og 7.	Godkendt
Punkt 3	Bilen bakker, indtil refleksbrik nummer 5 er passeret, og bringes til standsning inden refleksbrik nummer 4 nås.	Visuel test: Bilen bakker og standser mellem refleksbrikkerne 4 og 5.	Bilen bakker og standser mellem refleksbrikkerne 4 og 5.	Godkendt
Punkt 4	Bilen kører forlæns, indtil refleksbrik nummer 7 nås.	Visuel test: Bilen kører frem til refleksbrik nummer 7.	Bilen kører frem til refleksbrik nummer 7.	Godkendt
Punkt 5	Bilens bringes til standsning helt inde i mål-området, der er defineret som området mellem banens slut-ende og en meter efter banens afslutning.	Visuel test: Hele bilen står stille indenfor mål-området, der er defineret som en meter efter banens afslutning.	Hele bilen står stille indenfor mål-området, der er defineret som en meter efter banens afslutning.	Godkendt

Use Case 2: "Afspil lyd"	Test	Forventet resultat	Opnået resultat	Godkendt/ kommentar
Punkt 1	Use Case 1 ("Kør banen") udføres.	Akustisk test: Når bilen tændes for at køre ind på banen: Der afspilles en specifik "startlyd" eller "startmelodi".	Bilen afspiller den valgte startlyd.	Godkendt

Punkt 2	Hver gang en refleksbrik passeres, afspilles en specifik "refleksbriklyd".	Akustisk test: Når bilen passerer en refleksbrik, så vil bilen afspille en kanonskuds lyd.	Hver gang bilen passerer en refleksbrik, afspiller bilen en kanonskuds lyd.	Godkendt
Punkt 3	Når refleksbrik nummer 7 passeres, afspilles en specifik "slutlyd" eller "slutmelodi".	Akustisk test: Når bilen passerer den sidste refleksbrik, så skal bilen afspille en passende "slutlyd" til temaet.	Når bilen passerer den sidste refleksbrik afspiller bilen "slutlyden"	Godkendt

Use Case 3: "Styr forlys"	Test	Forventet resultat	Opnået resultat	Godkendt/ kommentar
Punkt 1	Forlys implementeres med 2 hvide LED-sæt, der monteres med et sæt i henholdsvis højre og venstre side.	Visuel test: Under kørsel skal forlys være tændt. Når bilen er stationær, skal forlysene være slukket.	Under kørsel er forlys tændt. Når bilen er stationær, er forlysene slukket.	Godkendt

Use Case 4: "Styr baglys"	Test	Forventet resultat	Opnået resultat	Godkendt/ kommentar
Punkt 1	Bag- og bremselys implementeres med 2 røde LED-sæt, der monteres med et sæt i henholdsvis højre og venstre side.	Visuel test: Under kørsel skal baglys være tændt. Når bilen bremser forøges lysstyrken og slukkes når den er stationær.	Under kørsel er baglyset tændt. Når bilen bremser forøges lysstyrken og slukkes når den er stationær.	Godkendt

Bilens tema	Test	Forventet Resultat	Opnået Resultat	Godkendt / kommentar
Punkt 1	Afgøres ved afstemning blandt reviewgruppens medlemmer.	Det forventes at størstedelen af gruppens medlemmer synes bilen ligner en tank.	Størstedelen af gruppens medlemmer synes at bilen ligner en tank.	Godkendt
Punkt 2	Den afspiller den rigtige fil for startmelodien.	Den afspiller den udvalgte lyd.	Bilen afspiller den rigtige 'startmelodi'.	Godkendt
Punkt 3	Den afspiller den rigtige fil for passering af refleksbrik.	Den afspiller den udvalgte lyd.	Bilen afspiller den rigtige 'reflekslyd'.	Godkendt
Punkt 4	Den afspiller den rigtige fil for slutmelodien.	Den afspiller den udvalgte lyd.	Bilen afspiller den rigtige 'reflekslyd'.	Godkendt

11.3. Accepttestspecifikation for ikke-funktionelle krav

Krav nr.	Krav	Test	Forventet resultat	Opnået resultat	Godkendt / kommentar
1.1	Bilen skal på en enkelt opladning af dennes batterier kunne gennemføre mindst 5 gennemkørsler af banen.	Vi lader batteriet helt op og tester hvorvidt bilen kan gennemføre 5 runder på banen.	Det forventes at batteriet har tilstrækkelig kapacitet til at gennemføre 5 runder på banen	Bilen kunne godt gennemføre 5 runder på banen.	Godkendt
1.2	På bilens højre og venstre side skal placeres detektorer, der kan registrere en R80 refleksbrik i afstanden 2 cm til 25 cm	Dette eftervises ved at LED'en lyser når refleksbrikken er placeret i intervallet 2 til 25 centimeter fra detektoren.	Ved brug af Analog Discovery eftervises det at detektoren kan registrere en refleksbrik, der er placeret i hele intervallet 2 til 25 cm fra detektoren.	Ved brug af Analog Discovery eftervises det at detektoren kan registrere en refleksbrik, der er placeret i hele intervallet 2 til 25 cm fra	Godkendt

				detektoren .	
1.3	Bilen monteret med al udstyr må maksimalt veje 5,2 kg	Bilen monteret med al udstyr vejes på en vægt med en målenøjagtighed bedre end 100 gram.	Vægten viser en vægt mindre end eller lig med 5,2 kg.	Vægten viser en vægt på 4615 g.	Godkendt
1.4	Bilens maksimale højde skal være 41,0 cm.	Bilen måles fra bund til top med et målebånd med 1 millimeters nøjagtighed.	Målebåndet viser en højde på mindre eller lig med 41,0 cm.	Målebåndet viser en højde på under 41,0 cm.	Godkendt
1.5	Bilen skal have en fungerende fejltæller. Fejltælleren skal inkludere en kugle som hænger på en snor. Når kuglen rører siden af fejltælleren, så tæller det som en fejl.	Fejltælleren testes i IDE.	Vores forventede resultat er at fejltælleren virker, og tæller 1 fejl per berøring	Vores forventede resultat er at fejltælleren virker, og tæller 1 fejl per berøring	Godkendt
2.1	Forlyset implementeres med 2 hvide LED-sæt, der monteres med et sæt i henholdsvis højre og venstre side.	Visuel test: Der er monteret 2 hvide LED-sæt på forenden af bilen.	Begge bilens LED'erne vil lyse så det kan ses med det blotte øje.	Begge bilens LED'erne lyser så det kan ses med det blotte øje.	Godkendt
2.2	Når forlyset er tændt, skal middelstrømmen gennem hver LED være 25 mA +/- 5 mA.	Et amperemeters pindes tilsluttes i serie på hver LED og dette vil blive afmålt til at være 25 mA +/- 5mA.	Amperemetret viser en værdi på 25 +/- 5 mA.	Amperemetret viser en værdi på 24,2 mA.	Godkendt
3.1	Bag- og bremselys implementeres med 2 røde LED-sæt, der monteres med et sæt i henholdsvis højre og venstre side.	Visuel test: Der er 2 røde LED-sæt monteret på bilens bagende.	Begge bilens LED'erne vil lyse så det kan ses med det blotte øje.	Begge bilens LED'erne lyser så det kan ses med det blotte øje.	Godkendt
3.2	Ved "bremselys" skal middelstrømmen gennem hver LED skal være 25 mA +/- 5 mA.	Et amperemeters pindes tilsluttes i serie på hver LED blive afmålt til at være 25 mA +/- 5mA.	Amperemetret viser en værdi på 25 +/- 5 mA	Amperemetret viser en værdi på 25.23 mA	Godkendt

3.3	Ved "almindeligt baglys" skal middelstrømmen gennem hvert LED være 5 mA +/- 2 mA.	Et amperemeters pindes tilsluttes i serie på hver LED og dette vil blive afmålt til at være 5 mA +/- 2mA.	Amperemetere t viser en værdi på 5 +/- 2 mA	Amperemetere t viser en værdi på 5.2 mA	Godkendt
4.1.2	Startlyden kan spille kontinuerligt i 2 sekunder (eller indtil melodien er slut)	Man trykker på startknappen, hvorefter startlyden kan afspilles kontinuerligt i 2 sekunder (eller indtil melodien er slut)	Vi forventer at når man trykker på startknappen afspilles startlyden i 2 sekunder eller til den er slut	Når man trykker på startknappen afspilles startlyden i 2 sekunder eller til den er slut	Godkendt
4.1.3	Startlydens niveau kan måles til 70dB på 5 meters afstand.	Med et dB-meter måles lydniveauet under afspilning.	Vi forventer at lyden afspilles ved det specificerede niveau.	Lyden afspilles ved det specificerede niveau.	Godkendt
4.2.2	Afspilning af reflekslyd bør påbegyndes indenfor 100ms af registrering af refleksbrik.	Vha. Analog Discovery måles tiden fra registrering til afspilning.	Vi forventer at lyden afspilles planmæssigt.	Afspilningen af lyd begynder efter 333 ms	Ikke-Godkendt
4.2.3	Reflekslydens niveau kan måles til 70dB på 5 meters afstand.	Med et dB-meter måles lydniveauet under afspilning.	Vi forventer at lyden afspilles ved det specificerede niveau	Lyden afspilles ved det specificerede niveau.	Godkendt
4.3.2	Slutlydens niveau kan måles til 70dB på 5 meters afstand.	Med et dB-meter måles lydniveauet under afspilning.	Det forventes at en decibelmeter viser mindst 70dB på 5 meters afstand.	Lyden afspilles ved det specificerede niveau.	Godkendt
4.3.3	Slutlyden må ikke afbrydes (eksempelvis af refleksbrik-lyden)	Slutlyden igangsættes og det konstateres om lyden afbrydes	Det forventes at lyden afspilles planmæssigt.	Lyden afspilles planmæssigt.	Godkendt

11. Konklusion

Udarbejdet af hele gruppen

11.1. Konklusion for hardwaredelen

Konstruktionen af al vores hardware har ikke været uden vanskeligheder; man går ind til et problem som virker meget simpelt (det kunne være konstruktion af veroboards til forlys, som eksempel) og tænker at det hurtigt kan gøres, men dårligt er man kommet i gang før de første problemer opstår.

Her er konfronteret man med virkeligheden at ens forståelse for emnet ikke er så god som man oprindeligt troede, og man tvinges derfor til at revurdere ens kompetencer, at genopsoge det relevante pensum, og herigennem faktisk at opnå en større forståelse. Denne iterative proces gentages om og om igen i hele processen, men med det udfald at vi alle har fået styrket vores ingeniørfaglige kompetencer hvad angår både analyse og konstruktion af hardware.

11.2. Konklusion for softwaredelen

Softwaren til bilen har lænet sig meget op ad MSYS-øvelserne og projektøvelserne, alligevel var der flere udfordringer i at skrive kodebasen til vores bil. Det skyldes blandt andet at omfanget af koden var større end hvad vi havde været vant til i de mere isolerede lab-opgaver vi har løst i MSYS. Derudover komplicerer det processen når flere mennesker er ansvarlige for hver deres del; på trods af en grundig forberedelse i vores software-design voldte det alligevel problemer at integrere software-modulerne. I sidste ende kom vi dog i mål og endte med en kode der nu eksekveres korrekt hver gang bilen testes på banen.

11.3. Konklusion for projektet

At gennemføre et projekt som dette i samarbejde med andre studerende med vidt forskellige baggrunde og arbejdsprocesser har været både udfordrende og lærerigt. I gruppen faldt vi hurtigt ind i en rytme hvor få personer i gruppen påtog sig et overordnet ansvar og havde et større overblik, hvilket tillod en effektiv arbejdsproces hvor beslutninger blev taget hurtigt og uden for meget unødigt diskussion. Generelt har gruppens dynamik fungeret rigtigt godt, selvom der dog har været en mindre asymmetrisk arbejdsfordeling.

Projektforløbet lykkes i høj grad med at binde semestrets fag sammen idet man bliver sat i situationer hvor man er nødt til at forstå elementer af både konstruktion og analyse af kredsløb samt digital styring af de kredsløb vha. egen kode skrevet til en microcontroller. Vi er alle kommet ud af forløbet med en øget forståelse for både det rent faglige indhold af semestret, men i høj grad også hvad det vil sige at læne sig op af hinandens kompetencer samt finde sig på plads i ens egen rolle som del af en større sammenhæng.

12. Individuelle konklusioner

Frederik:

I udfærdigelsen af vores projekt har jeg lært en hel del om hvordan man konstruerer kredsløb samt hvilke overvejelser man skal gøre sig herom. Det har været meget udfordrende til tider, især fordi det kræver en forståelse og en måde at tænke på som er uvant for mig. Noget af det mest overraskende i den rent faglige proces har været at erfare hvordan enkeltstående moduler kan fungere fejlfrit, men at integrationen af dem til trods for dette kan være meget vanskelig.

Som koordinerende observatør har jeg en meget logisk og informationssøgende tilgang til problemer, hvilket ofte er på bekostning af hurtige beslutninger. Jeg har til tider måtte tilsidesætte egne præferencer til fordel for at vi som gruppe kunne komme fremad, og jeg har været heldig at andre i gruppen netop har stærkere kompetencer som beslutningstager end jeg selv. Det vil fortsat være et arbejds punkt for mig, både i fremtidige projekter på studiet, men sikkert også i forbindelse med arbejde og andet.

Oskar:

Rent fagligt har der været meget tydelige forbindelser mellem semesterprojektet og alle kurserne på 1. semester. I stedet synes jeg, at mine største problemer har været alle de opgaver, der lå forud for det selvstændige projektarbejde. Jeg har ikke været vant til at udarbejde en kravspecifikation, lave diagrammer, og samarbejde med så mange forskellige mennesker. Arbejdet med motor og sensor i både HW og SW var langt mere overskueligt, selvom det heller ikke var helt uden problemer. Det falder mig naturligt at fordybe mig, og arbejde selvstændigt, og netop derfor har det sværeste for mig, været de større fælles indsatser. Jeg har til tider været mindre kompromisøgende, og gruppeorienteret, og i stedet lavet opgaver fremfor at uddele dem.

Det hænger godt sammen med min blå og røde personlighedstype, og er helt sikkert noget jeg fremadrettet skal arbejde med. Forbedre den fælles kommunikation, og minimere mit behov for at lægge sidste hånd på værket. På mange måder har jeg lært mere om projektarbejde, hvordan forskellige mennesker arbejder sammen, og hvordan jeg selv arbejder - end selve det faglige.

Andreas:

Semesterprojektet har været lærerigt og har vist mig sammenhængen mellem teori og praksis. Jeg har oplevet, hvor udfordrende det kan være at integrere forskellige moduler til et fungerende system.

Som person med en grøn insights profil har mine styrker været min rolige og støttende tilgang til gruppearbejdet. Jeg har bidraget med stabilitet og været god til at lytte til andres ideer, samt hjælpe med at skabe enighed i gruppen når der opstod uenigheder.

Min største svaghed har været mangel på initiativ og en tendens til at vente på, at andre træffer beslutninger i stedet for selv at tage føringen. Min grønne profil betyder, at jeg foretrækker at analysere grundigt før jeg handler, men det har til tider bremsset gruppens fremdrift.

Fremadrettet vil jeg arbejde på at være mere proaktiv og udtrykke mine synspunkter tydeligere, så mine analytiske evner kan komme gruppen til gavn på en mere aktiv måde.

Morten:

I arbejdet med semesterprojektet har jeg tilegnet mig ny og værdifuld viden. Selvom det undervejs kunne være svært at se sammenhængen mellem fagene og projektet, gav det hele mening i de afsluttende uger, hvor vi arbejdede med implementeringen af hardware og software. Det har været en meget lærerig proces og givet mig en bedre helhedsforståelse for projekter af denne størrelse.

Jeg har i samarbejde med Frederik og Tim haft ansvar for LED-enheden til bilen og har stået for kodning og opsætning med Arduino. Derudover har jeg stået for bilens udsmykning, som skulle ligne en tank. I projektets opstart var jeg ikke den, der tog teten, men støttede op, hvor det var nødvendigt – hvilket passer med min Insights-profil som "Hjælpende Supporter". Fremadrettet vil jeg arbejde på at skabe et bedre overblik fra start, da dette til tider var en udfordring for mig i dette projekt.

Viktor:

Projektet har givet betydelig læring, især gennem selvstudie, da jeg ikke synes undervisningen altid var tilstrækkelig. Dette har styrket min selvhjulpethed. Det var særligt givende at se, hvordan teorien fra Microcontroller systemer fandt praktisk anvendelse i min udarbejdelse af lyden.

Mine styrker i gruppen var mit initiativ, min motivation af gruppen og min positive indstilling, som bidrog til et godt arbejdsmiljø. Disse stemmer overens med min Insights-profil (primært rød og gul), der afspejler min rolle som Entreprenør/Inspirator, hvor jeg trives med at tænke nyt og inspirere.

Min største svaghed var inden for hardware, hvor jeg fandt de komplekse aspekter udfordrende og måtte læse meget op. Jeg følte mig ikke altid fuldt kompetent på dette område. Min røde/gule profil, som fokuserer på de store linjer, kan have bidraget til at overse tekniske detaljer. Fremadrettet vil jeg aktivt forbedre mine hardwarekunderskaber og min detaljeorientering for at opbygge en dybere grundlæggende forståelse og bidrage mere effektivt i fremtidige projekter.

Mikkel:

Semesterprojektet har været lærerig og spændende, samt har de tydelige forbindelser mellem det man lærer i kurserne og projektet, hjulpet mig med at forstå hvad man laver og hvordan det skal laves. Samt udfordringen ved at få tingene til at virke og forstå hvad der skal til, for at det virker, har givet virkelig meget og har været med til at gøre projektet godt, og gør at jeg kan føle mig stolt af hvad vi har lavet.

Min største svaghed har været at jeg var nybegynder indenfor elektronik, kodning osv. Inden jeg startede på uddannelsen, hvor andre har lavet eller arbejdet med tingene førhen. Da jeg lærer bedst gennem praksis, har det dog hjulpet at sidde med det, og få tingene forklaret af andre, så jeg stadig kan følge med dem. Udover dette skal jeg fokusere på at tage mere initiativ, i stedet for at vente på at jeg får uddelt en opgave.

Min styrke i gruppen har været at jeg arbejder roligt og grundigt, og at jeg er god til at støtte op om de andres ideer og beslutninger. Jeg bidrager med stabilitet og er altid klar til at hjælpe, hvis folk har brug for det. Dette passer til min grønne- og blå insightsprofil, hvor jeg trives med ro, struktur og samarbejde.

Tim:

Det har været en lærerig proces, og især har det været spændende at se sammenhængen mellem teorien og praksis. Jeg har lært meget om alt det arbejde der er før man begynder på selve produktet, som at udarbejde en accepttest, kravspecifikation og de forskellige diagrammer. Og hvor meget det kan hjælpe en at have noget at holde produktet op imod gennem de forskellige stadier samt til sidst. Selvom det ikke kom helt bag på mig at når vi skulle samle alle delene ville vi få problemer selvom de virkede hver for sig, var det svære at forstå hvorfor det skete.

Min svaghed er at jeg ikke er den der tager kontrol, men er mere tilbageholden og bare indfinder mig med hvad gruppen holdning og planer er.

Min styrke er at arbejde struktureret, være objektiv og analyserende. Jeg er ikke personen der er mest kreativ eller opfindsom, men er god til at se hvad andre overser og arbejde efter en plan. Dette stemmer godt overens med min grønne og blå insightprofil.

13. Bilagsoversigt

- Tidsplan
- Mødedeltagelseskema
- Samarbejdskontrakt
- Logbog
- Vejledermøder
- LAB-Øvelser
- Datasheets